1. Deloitte — Azure Data Engineer / Databricks

Q1. What is the Medallion Architecture and why is it used in Azure Databricks projects?

Answer:

Medallion architecture splits data into Bronze, Silver, and Gold layers. Bronze holds raw ingested data, Silver has cleaned and conformed data, and Gold contains business -ready aggregates and KPIs. Deloitte -type consultin g projects use this to make pipelines easier to debug, reprocess, and govern across multiple clients and use cases.

Q2. Why is Delta Lake preferred over plain Parquet in enterprise projects?

Answer:

Delta Lake adds a transaction log (_delta_log) on top of Parquet, which enables ACID transactions, schema enforcement, and time travel. This means you can do reliable UPSERTs, DELETEs, and MERGE operations at scale without corrupting tables. In consulting environments with frequent changes and multiple teams, t his reliability is non -negotiable.

Q3. When do you use Z -ORDER in Databricks, and how does it help?

Answer:

Z-ORDER is used when you frequently filter on one or more high -selectivity columns like customer_id, order_id, or a date range. It physically reord ers data files so that related values are stored close together, which drastically improves data skipping and reduces I/O. This becomes important when your table is in the hundreds of GBs or TBs and query latency matters.

Q4. How does Auto Loader improve ingestion compared to ADF Copy Activity?

Answer:

Auto Loader uses a file notification or incremental listing mechanism to detect only new files instead of re -scanning entire folders. It supports schema inference, schema evolution, and

exactly -once semantics via checkpoints, which makes it ideal for continuous ingestion. ADF Copy is better for simple, batch, one -off or low -frequency loads rather than large -scale streaming -like scenarios.

Q5. What is Unity Catalog and why is it important for a Deloit te-style multi -client setup?

Answer:

Unity Catalog centralizes governance, permissions, and lineage across all workspaces and clusters. Instead of managing permissions per workspace or table manually, you define them once via catalogs, schemas, and tables. This fits perfectly for service -based firms like Deloitte where multiple projects, teams, and clients share the same platform but must be securely isolated.

Q6. How would you handle personally identifiable information (PII) in Databricks?

Answer:

You typ ically hash or tokenize sensitive columns (like email, phone, PAN) in Silver and Gold while keeping the raw version only in tightly restricted Bronze. Row/column -level security can be enforced using dynamic views and Unity Catalog policies. In some cases, you combine hashing with salting or reversible tokenization depending on whether you need exact matching or reversibility.

Q7. What causes “small file problem ” and how do you fix it?

Answer:

Small files are generated when many micro -batches write tiny chu nks of data or when code over-partitions output. This kills performance because Spark has to open and scan too many files. You fix it with Auto Optimize, manual OPTIMIZE + ZORDER, and better partitioning/repartitioning strategies before writing to Delta.

Q8. How do COPY INTO and Auto Loader differ in usage?

Answer:

COPY INTO is best for batch -style loading from static file sets or when you want to load historical or backfill data in chunks. Auto Loader is built for continuous / streaming style ingestion wh ere new files keep arriving, and you want incremental processing with checkpointing and schema evolution. In a well -designed system, you often use COPY INTO for one -time backfill and Auto Loader for ongoing loads.

Q9. How does Delta Lake implement ACID tr ansactions under the hood?

Answer:

Delta Lake maintains a transaction log in _delta_log with a new JSON (and parquet) file per commit. Writers use optimistic concurrency, meaning they check that the files they read weren ’t modified by others before they co mmit changes. Readers always reconstruct a table snapshot from a consistent log version, giving snapshot isolation and full ACID guarantees even with multiple concurrent writers.

Q10. How would you design a CDC pipeline from an OLTP system into Azure Data bricks?

Answer:

You extract changes from the source system using tools like ADF, Debezium, or native CDC features and land them into a Bronze Delta table with operation flags (I/U/D). In Silver, you apply business rules, deduplication, and type -casting. In Gold, you maintain SCD2 or flattened tables using Delta MERGE, possibly leveraging Change Data Feed (CDF) for efficient incremental processing.

Q11. What is Adaptive Query Execution (AQE) in Spark and why is it useful?

Answer:

AQE adjusts the physical execution plan at runtime based on actual data statistics. It can change join types (e.g., to broadcast), coalesce small partitions, and handle skewed data more

gracefully. This makes big queries more robust and performant, especially in consulting environ ments where data distributions differ heavily between clients or environments.

Q12. How do you debug skewed joins in Spark?

Answer:

You first inspect the Spark UI to see if certain tasks are taking much longer and if one partition has disproportionate dat a. Then you identify skewed keys by profiling the data distribution. To fix it, you can broadcast the smaller table, salt the skewed key (add random suffixes), or use AQE with skew join enabled so Spark rewrites the plan dynamically.

Q13. What is checkpoi nting in Structured Streaming and when is it mandatory?

Answer:

Checkpointing stores the streaming query ’s progress (offsets) and any state needed for aggregations or joins. It is mandatory when you need exactly -once or at -least-once guarantees and want to handle failures gracefully without reprocessing everything from scratch. Without checkpoints, a restart would behave like a new job with no awareness of past progress.

Q14. Why do we keep Bronze as append -only and avoid deletes/updates there?

Answer:

Bronze is your source -of-truth raw history, so it must reflect exactly what came from the source, including bad data or duplicates. This makes debugging and reprocessing much easier since you can always reconstruct higher layers. If you start “fixing ” Bronze, you lose forensic capability and complicate audits.

Q15. How do you design SCD Type -2 logic in Databricks using Delta MERGE?

Answer:

You keep columns like effective_from, effective_to, and is_current in the dimension table. During MERGE, you expire the c urrent record (is_current = false, set effective_to) and insert

a new one with updated values and is_current = true. The natural key (e.g., customer_id) plus is_current acts as the join condition to find the active record.

Q16. What is Data Skipping and h ow does Delta Lake use it?

Answer:

Data Skipping uses stored statistics like min/max values for each column per file to eliminate scanning irrelevant files. When a query filters on a column, Delta checks these stats and skips files that cannot contain matc hing rows. This is critical when you have thousands of files and need fast selective queries.

Q17. Why is Unity Catalog preferred over legacy Hive Metastore?

Answer:

Unity Catalog adds centralized governance, fine -grained permissions, lineages, and audit logs across workspaces. Hive Metastore is limited to table metadata and doesn ’t give modern governance or cross -workspace visibility. For a large consulting firm, UC allows standardized security policies and simplifies compliance massively.

Q18. How would you build a near real -time ingestion architecture on Azure for Deloitte ’s client?

Answer:

Use Event Hubs or Kafka for streaming source, Auto Loader for ingesting into Bronze, and Structured Streaming notebooks for Silver. Gold aggregates update either in streaming mode for low -latency KPIs or in small micro -batches. You orchestrate with Databricks Jobs / Workflows or ADF, and serve dashboards via Power BI/SQL Warehouse.

Q19. Deloitte SQL Question – Top 3 Departments by Avg Salary

Question: Get the top 3 departments by average salary.

SELECT department,

A VG(salary) AS avg_salary

FROM employees

GROUP BY department

ORDER BY avg_salary DESC

FETCH FIRST 3 ROWS ONLY;

Q20. Deloitte PySpark Question – Keep Latest Record Per ID

Question: Deduplicate a tabl e and keep only latest record by timestamp per id.

```
from pyspark.sql import functions as F, Window
w = Window.partitionBy("id").orderBy(F.col("ts").desc())
df_latest = (df
.withColumn("rn", F.row_number().over(w))
.filter(F.col("rn") == 1)
.drop("rn")
)
```

2. Amazon — Data Engineer (Product -Based, with DSA)

Q1. How would you design a large -scale orders data pipeline at Amazon?

Answer:

You’d ingest order events through Kinesis or Kafka from multiple microservices. These are streamed into S3 or A DLS using Spark Structured Streaming or Kinesis Data Analytics, landing as Bronze. Then you build Silver/Gold tables for order status, revenue, and fulfillment metrics, serving Redshift or Databricks SQL for downstream analytics.

Q2. Explain event -time vs processing -time and why it matters at Amazon scale.

Answer:

Event -time is the actual time the event occurred at the source (e.g., when user clicked “buy”).

Processing -time is when your system processes that event. At Amazon ’s volume, network delays and retries make them diverge; using event -time + watermarks ensure s windows and metrics reflect real user behavior, not temporary processing delays.

Q3. How do you design for at -least-once vs exactly -once semantics?

Answer:

At-least-once means duplicates can occur but no event is lost, which is acceptable if downstream writes are idempotent. Exactly -once requires deduplication or transactional sinks so that each event affects state only once. In practice, Amazon often uses at -least-once ingestion with idempotent upserts in Delta or DynamoDB to achieve effective exactly -once behavior.

Q4. What is watermarking in Structured Streaming and how is it used for late data?

Answer:

Watermarking lets Spark keep state only for a limited event -time range and drop older state beyond that. For example, with a 2 -hour watermark, late events older than 2 hours are ignored or sent to a dead -letter stream. This prevents unbounded state growth while still handling reasonable late -arrival behavior, which is common in large distributed systems.

Q5. How would you partition a multi -terabyte o rders fact table?

Answer:

Typically you partition by order_date or order_month and possibly region or marketplace. The partitioning must align with common query filters while avoiding too many small partitions. High -cardinality fields like customer_id are bad partition keys and instead are better candidates for ZORDER or data skipping.

Q6. What is backpressure in streaming systems and how does Spark handle it?

Answer:

Backpressure is when the system cannot process data as fast as it ’s arriving, causing l ag. Spark can adjust batch sizes or trigger interval to match processing capabilities, and you can scale out clusters or optimize code to reduce bottlenecks. If ignored, backpressure leads to SLA failures and potential data loss in extreme cases.

Q7. How do you reduce heavy shuffling in Spark jobs?

Answer:

You reduce unnecessary repartition and wide transformations like large groupBy or joins. Preferring broadcast joins, bucketing, and pre -partitioned data on join keys massively lowers shuffle volume. Als o tuning spark.sql.shuffle.partitions to a realistic value for your cluster size avoids over/under -partitioning.

Q8. Compare ADLS and S3 from a data engineering perspective.

Answer:

Both are object stores, but ADLS Gen2 supports a hierarchical namespace and POSIX -compatible ACLs out -of-the-box. S3 integrates tightly with AWS ecosystem and uses IAM policies. Functionally they serve similar roles for data lakes, but operational details and security models differ across Azure vs AWS.

Q9. What is Kinesis and where would you use it?

Answer:

Kinesis is Amazon ’s managed streaming platform, similar to Kafka or Event Hubs. It ’s used for high -throughput ingestion of logs, clickstreams, metrics, and real -time events. You typically consume from it with Kinesis Analy tics, Spark Streaming, or custom consumers to feed data lakes and real -time processing pipelines.

Q10. How do you optimize Redshift Spectrum or external table queries on a data lake?

Answer:

Store data in columnar formats like Parquet, partition based on query patterns (date/region), and compress using Snappy/ZSTD. Ensure your external schema structure follows partition columns and that you keep file sizes in a few hundred MB range. This reduces scan size and improves both cost and latency.

Q11. What is file compaction and why is it important in Delta tables?

Answer:

Compaction merges many small files into fewer larger ones, typically 128–512 MB each. This reduces file listing overhead and improves scan performance dramatically. It ’s essential in streami ng environments or high -frequency writes where small files naturally accumulate.

Q12. How does Delta Lake differ from a managed warehouse like Redshift?

Answer:

Delta Lake sits on a data lake and gives ACID + flexible compute, letting you plug in Spark, SQL, ML, streaming from the same tables. Redshift is a managed MPP warehouse optimized purely for SQL analytics and structured workloads. At Amazon scale you often see both: Delta/Spark for heavy ETL/ML; Redshift/Spectrum for BI.

Q13. When would you use D ynamoDB in a data engineering solution?

Answer:

DynamoDB is ideal when you need low -latency key -value lookups or simple queries with predictable performance. Typical use cases include session stores, materialized aggregates keyed by ID, or fast online loo kups in microservices. It ’s not for complex ad -hoc analytics; that’s what Redshift/Delta/S3 are for.

Q14. What is AWS Glue Data Catalog and how does it relate to Databricks or Spark?

Answer:

Glue Catalog is AWS ’s central metastore holding table and schema definitions on top of S3

data. Spark, EMR, and Athena can all use this catalog, making data discoverable across services. It plays a similar role to Hive Metastore or Unity Catalog (to some extent) in the Azure/Databricks ecosyste m.

Q15. Why would Amazon use bucketing on large fact tables?

Answer:

Bucketing pre -hashes data on a column into a fixed number of buckets. When you join two bucketed tables on the same key and bucket count, Spark can avoid full shuffles, massively speedi ng up joins. It ’s especially useful when joining giant fact tables repeatedly in a consistent way.

Q16. Amazon SQL Question

For each calendar month, find the top 3 products by net revenue, where:

Gross revenue = SUM(quantity * unit_price) from order_ite ms, for orders with status = 'COMPLETED'.

Refunds = SUM(refund_amount) from returns.

Net revenue = gross_revenue - refunds (if no refund, treat as zero).

Consider only products with net_revenue > 0.

Output: month_start, product_id, net_revenue, rank_in _month (1 = highest).

Use SQL (Redshift/Postgres -style) with window functions and multiple joins.

Answer / Solution

```
WITH completed_orders AS (
SELECT
o.order_id,
date_trunc('month', o.order_date)::date AS month_start
FROM orders o
```

```
WHERE o.status = 'COMPLETED'

),

gross_revenue AS (

SELECT

co.month_start,

oi.product_id,

SUM(oi.quantity * oi.unit_price) AS gross_revenue

FROM completed_orders co

JOIN order_items oi

ON co.order_id = oi.orde r_id

GROUP BY co.month_start, oi.product_id

),

refunds AS (

SELECT

co.month_start,

r.product_id,

SUM(r.refund_amount) AS total_refund

FROM completed_orders co

JOIN returns r

ON co.order_id = r.order_id

GRO UP BY co.month_start, r.product_id

),

net_revenue AS (

SELECT
```

```
g.month_start,
g.product_id,
g.gross_revenue
- COALESCE(r.total_refund, 0) AS net_revenue
FROM gross_revenue g
LEFT JOIN refunds r
ON g.month _start = r.month_start
AND g.product_id = r.product_id
WHERE g.gross_revenue - COALESCE(r.total_refund, 0) > 0
),
ranked AS (
SELECT
month_start,
product_id,
net_revenue,
RANK() OVER (
PARTITION BY month_start
ORDER BY net_revenue DESC
) AS rank_in_month
FROM net_revenue
)
SELECT
month_start,
product_id,
```

net_revenue,

rank_in_month

FROM ranked

WHERE rank_in_month <= 3

ORDER BY month_start, rank_in_month, product_id;

Q17. Amazon PySpark Question – 7-Day Rolling Sum Per Customer

Question: Compute 7 -day rolling sum of amount per customer ordered by date.

from pyspark.sql import functions as F, Window

w = (Window

.partitionBy("customer_id")

.orde rBy("order_date")

.rowsBetween( -6, 0))

df_with_roll = df.withColumn("rolling_7d_amount", F.sum("amount").over(w))

DSA 1 – K Closest Points to Origin

Answer:

Use a max -heap (priority queue) of size k. Iterate through the points, compute distance squared, and push to heap; if heap size exceeds k, pop the largest. At the end, the heap contains the k closest points in O(n log k) time, which is efficient for large n.

DSA 2 – Longest Substring Without Repeating Characters

Answer:

Use a sliding window wi th two pointers (left, right) and a hashmap of last seen positions for each character. Move right forward and shrink left whenever a duplicate is found within the current window. This gives you O(n) time complexity and is a standard Amazon -style string question.

DSA 3 – Merge Overlapping Intervals

Answer:

First sort intervals by start time. Then scan from left to right, merging intervals when the current interval ’s start is ≤ the last merged interval ’s end. This greedy approach is O(n log n) due to sorting and is useful for schedule merging, range compression, or compaction of availability windows.

3. PwC — AZURE Data Engineer

Q1. Why is data quality so critical in consulting projects like PwC ’s?

Answer:

Clients often use the data for financial re porting, compliance, or executive dashboards, so wrong numbers directly damage trust. Poor quality at Bronze/Silver means weeks of rework, escalations, and loss of credibility. That ’s why you implement explicit DQ rules (completeness, uniqueness, referenti al integrity) and log every failure.

Q2. What is the difference between batch and micro -batch processing?

Answer:

Batch means you process large chunks of data at fixed intervals (daily, hourly). Micro -batch in Structured Streaming still uses batches but they’re very small and frequent, giving near real-time processing while retaining Spark ’s batch engine semantics. PwC frequently uses micro -batch to balance latency and operational simplicity.

Q3. How would you define audit columns in enterprise tables?

Answer:

You typically add created_ts, updated_ts, source_system, batch_id, and sometimes file_name. These allow you to trace exactly when and how a record arrived and was modified. It helps with debugging, reproducing issues, and complying with audit requi rements.

Q4. How does GDPR deletion work in Delta?

Answer:

You identify records to delete based on keys like customer_id or other identifiers. Then you use a DELETE operation or MERGE into a Delta table to logically remove them, which updates the transac tion log. Physical data is removed only after VACUUM runs and retention period is over, giving you a controlled deletion process.

Q5. What are the main Structured Streaming trigger types?

Answer:

Trigger.Once processes all available data once and then st ops, good for incremental batch patterns. processingTime (e.g., every 1 minute) runs micro -batches periodically. Continuous mode has ultra -low latency but more restrictions and is less commonly used in conservative consulting environments.

Q6. Why is Bronze usually append -only and not mutated?

Answer:

Bronze must preserve the original fact of what the source provided, good or bad. If you mutate Bronze, you lose the ability to reprocess and investigate problems. All corrections, enrichments, and busine ss logic should happen at Silver or Gold so you maintain a clean lineage.

Q7. Compare Snowflake and Databricks from a PwC data engineering perspective.

Answer:

Snowflake is mainly a warehouse with strong SQL, elasticity, and simplicity, great for BI - heav y workloads. Databricks is a general compute platform with Spark, ML, streaming, and Delta Lake, ideal for complex transformations and data science. Many PwC projects combine them: Databricks for heavy ETL/ML, Snowflake for serving analytics.

Q8. What doe s “orchestration ” mean in an Azure DE context?

Answer:

Orchestration is coordinating all ETL steps: triggers, dependencies, retries, alerts, and environment -specific parameters. Tools like ADF or Databricks Workflows handle which job runs after which, how failures are handled, and how SLAs are tracked. Without orchestration, pipelines turn into unmanageable spaghetti.

Q9. How would you design a multi -region DR strategy for a critical analytics platform?

Answer:

You replicate ADLS data to a secondary regi on and maintain metadata sync for Delta tables. Databricks workspaces or clusters in secondary region must be ready to attach to replicated storage. DNS or routing rules allow failover so that jobs and dashboards can switch to the secondary region with min imal downtime.

Q10. Why use service principals instead of user identities for pipelines?

Answer:

Service principals represent non -human application identities, so they ’re not tied to individual employees leaving the company. They can be granted least -privilege RBAC and used for automated jobs without interactive sign -in. This is far cleaner and more compliant than running production jobs with personal accounts.

Q11. How would you optimize Silver layer transformations?

Answer:

You cache frequently reused datasets and broadcast small reference tables or dimensions. You design transformations to minimize shuffles and reuse partitioning layouts from Bronze where logical. You also avoid unnecessary wide operations and do column pruning early so you don ’t carr y junk through the pipeline.

Q12. What are cluster pools and why might PwC use them?

Answer:

Cluster pools keep a set of pre -warmed nodes ready for fast cluster startup. When a job needs a new cluster, it picks nodes from the pool instead of provisioning from scratch, which saves a lot of time for short or frequent jobs. Consulting projects with many scheduled jobs benefit massively from this.

Q13. How does ACID in Delta help when multiple teams are writing concurrently?

Answer:

With ACID, writes either fully succeed or fully fail; there is no partial corruption of data. Snapshot isolation ensures readers never see half -written data. This allows multiple teams or jobs to safely share Delta tables without stepping on each other.

Q14. What is VACUUM in De lta and when should you be careful using it?

Answer:

VACUUM physically deletes data files that are no longer needed according to the transaction log and retention rules. You must be careful not to use too small a retention period, or you risk breaking tim e travel and long -running readers. It ’s typically run in off -peak hours and after validating backup/DR strategy.

Q15. How do you detect schema changes from the source?

Answer:

Auto Loader can detect new columns and log schema evolution events. You can al so store expected schema in a metadata table and compare against inferred schema. PwC -style production systems will usually alert on any schema drift and require explicit approval before propagating changes to Silver/Gold.

Q16. Why is star schema still co mmonly used in enterprise analytics?

Answer:

Star schema simplifies BI by separating facts (measurable events) from dimensions (descriptive attributes). It is easy for analysts to understand and for tools like Power BI to optimize. Even in lakehouse setup s, star schemas are still used in Gold for reporting efficiency and clarity.

Q17. What is the difference between row -based and column -based storage?

Answer:

Row -based storage is better for OLTP because it quickly writes and reads entire rows. Columnar formats like Parquet are better for analytics since queries often touch only a subset of columns and benefit from heavy compression. Delta uses columnar storage under the hood and adds ACID semantics on top.

Q18. How would you handle late -arriving data in PwC pipelines?

Answer:

You define a window of tolerance (e.g., late by up to 2 days) and use MERGE in Silver to update affected partitions. Gold tables are recomputed incrementally only for partitions touched by late records. Extremely late data could be routed to a separate correction process, depending on business rules.

Q19. PwC SQL Question – Employees Above Department Average

```
SELECT e.name,
e.salary,
e.dept
FROM employees e
JOIN (
SELECT dept,
```

A VG(salary) AS avg_sal

FROM employees

GROUP BY dept

) d

ON e.dept = d.dept

WHERE e.salary > d.avg_sal;

Q20. PwC PySpark Question – Explode Array Column

from pyspark.sql import functions as F

df_skills = df.withColumn("skill", F.explode("skills"))

4. KPMG — AZURE Data Eng ineer

Q1. How do you make sure pipelines meet SLAs for KPMG clients?

Answer:

You define clear start/end time expectations per job and track them via run -log tables.

Monitoring tools send alerts if durations exceed thresholds or if jobs fail. Over time you tune

cluster sizes, parallelism, and incremental logic to ensure consistent SLA compliance.

Q2. How do you handle multi -tenant data in the same Databricks environment?

Answer:

You separate data by catalog/schema per client, and often use separate sto rage containers per

tenant. Unity Catalog controls access so one client cannot see another client ’s data. You may

also use different workspaces or clusters when strict isolation is required.

Q3. What is “idempotent ETL ” and why does it matter?

Answer:

Idempotent ETL means running the same job multiple times on the same input doesn ’t

corrupt or duplicate data. This usually involves using MERGE, partition overwrites, or

deduplication logic keyed on batch IDs. It ’s essential because real production systems f ail and you will re -run jobs frequently.

Q4. How would you parameterize Databricks jobs for different regions or clients?

Answer:

You pass parameters like client_id, region, or env into notebooks or jobs, often via ADF or Workflows. Inside the notebook, you read them with widgets or job parameters and use them to look up config from metadata tables. This lets one codebase serve many clients.

Q5. Why is surrogate key generation important in KPMG data models?

Answer:

Surrogate keys provide a stable, inter nal identifier even when business keys change or differ between systems. They support SCD2, unified dimensions, and easier joins across multiple sources. They also protect against leaking business -sensitive identifiers to external consumers.

Q6. What is t he difference between data drift and schema drift?

Answer:

Schema drift is when the structure changes (columns added, removed, or types changed). Data drift is when the distribution of values shifts without structural changes, like average transaction amo unts suddenly increasing. Both must be monitored, but schema drift often breaks pipelines, whereas data drift silently breaks analytics or models.

Q7. Why is medallion architecture especially useful in audit -heavy industries?

Answer:

It provides clear boundaries between raw, cleaned, and business -ready data. Auditors can inspect each layer to see how data evolved and verify transformations. If something goes wrong at Gold, you can roll back and recompute from lower layers without losing source

informati on.

Q8. What is the role of Delta expectations or DQ checks in KPMG pipelines?

Answer:

These rules enforce constraints like NOT NULL, allowed ranges, uniqueness, and referential integrity. Records that fail checks can be quarantined or flagged for manual review. This ensures clients never consume unvalidated data in their regulatory or financial reports.

Q9. How does OPTIMIZE with ZORDER help long -running KPMG projects?

Answer:

OPTIMIZE compacts many small files into larger ones, and ZORDER BY (key) clu sters related rows together. This leads to less I/O and faster queries, especially for selective filters. It keeps performance predictable even as tables grow over time.

Q10. How do you detect duplication across batches?

Answer:

You compute a hash or com posite key of important fields and track those in a dedup table or in the target itself. New batches are joined against existing records, and duplicates are either ignored or flagged. This is common in file -based ingestion where upstream sometimes resends data.

Q11. What are DLT expectations and how do they help consulting teams?

Answer:

DLT (Delta Live Tables) expectations are declarative rules attached to tables that specify allowed values or constraints. They can drop, quarantine, or warn on bad record s automatically. This encodes DQ rules next to data pipelines and makes them easier to maintain and audit.

Q12. Why is metadata -driven ETL valuable for KPMG?

Answer:

It lets you onboard new sources/entities by changing rows in a metadata table instead of writing new code. ETL steps like column mapping, file paths, and validation rules are all defined as metadata. This makes solutions scalable across many clients and cheaper to maintain.

Q13. How do you handle incremental loading from SAP/Oracle in KPMG projects?

Answer:

Use CDC timestamps, high -water -mark columns, or log tables to fetch only changed rows. Land them into Bronze tagged with batch IDs, then MERGE into Silver/Gold using appropriate keys. For very high volumes, use CDF or log -based CDC tools to avoid full -table scans.

Q14. What is the purpose of Silver -to-Gold MERGE?

Answer:

Silver often stores the latest cleaned version of each business entity, while Gold applies aggregation or SCD logic. MERGE from Silver to Gold allows you to apply incre mental changes without rebuilding everything. This keeps pipelines efficient and incremental instead of full refresh.

Q15. How do you monitor data pipeline health over time?

Answer:

You track metrics like record counts, error counts, lag, SLA breaches, a nd resource usage. These are written to monitoring tables and visualized in dashboards. Alerts are configured to fire when thresholds are crossed, giving early warning before business users complain.

Q16. What ’s the difference between coalesce and reparti tion in Spark?

Answer:

coalesce(n) reduces the number of partitions without a full shuffle, usually after filtering data down. repartition(n) can increase or decrease partitions and always triggers a shuffle, giving a more even distribution. Coalesce is c heaper but less flexible than repartition.

Q17. When would you choose broadcast hash join vs sort -merge join?

Answer:

You choose broadcast join when one side is small enough (typically < 500 MB) to fit in memory on all executors, giving very fast joins. Sort-merge join is used when both datasets are large and need to be shuffled and sorted before join. KPMG -type large fact + small dimension patterns commonly use broadcast joins.

Q18. What is a Lakehouse and why is it relevant for KPMG?

Answer:

Lakehouse combines the scalability and flexibility of a data lake with the governance and performance of a warehouse. With Delta Lake and Databricks, you can run ETL, BI, and ML on the same data without moving it. This reduces duplication and speeds up delivery of analytics solutions for clients.

Q19. KPMG SQL Question – Second Highest Salary

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q20. KPMG PySpark Question – Null Ratio Per Column

```
from pyspark.sql im port functions as F
total_count = df.count()
null_stats = (df
.select([
```

```
(F.count(F.when(F.col(c).isNull(), c)) / F.lit(total_count)).alias(c)
for c in df.columns
])
)
null_stats.show()
```

5. Cognizant – Azure Data Engineer

Q1. Ho w do you decide partitioning strategy for large Delta tables?

Partitioning depends on filter frequency, cardinality, and future growth. You avoid columns like customer_id (high cardinality), but pick date or region because they minimize unnecessary file s canning. In Cognizant projects where dashboards query daily data, partition -by-date is almost always safer and more performant.

Q2. How would you design a metadata -driven ingestion framework?

Store ingestion configs (path, schema, delimiter, load type) in a Delta metadata table. A generic notebook reads metadata and applies ingestion logic dynamically without hardcoding. This reduces maintenance and allows onboarding new tables by simply inserting config rows, not deploying new code.

Q3. Why is Silver l ayer essential even if you have clean CDC feeds?

CDC feeds still contain missing values, wrong datatypes, duplicates, and unexpected nulls. Silver standardizes schema, validates mandatory fields, and enforces primary -key level uniqueness. This ensures Gol d layer logic is predictable and prevents inconsistent KPIs across Cognizant client dashboards.

Q4. How do you handle invalid records in ingestion pipelines?

Create a quarantine table storing rejected rows with error_code, error_description, and batch_id . This makes debugging easier and allows business teams to inspect or correct bad

records. You also track percentages of rejected rows over time to detect upstream problems.

Q5. What is optimized auto -compaction in Databricks?

When enabled, the platform automatically merges small files into larger ones as data is written. This prevents small -file explosion and keeps underlying storage efficient. Cognizant especially benefits from this because many pipelines run micro -batches generating many tiny output fi les.

Q6. Explain transaction log architecture of Delta Lake.

Each commit writes a JSON log with operations (add/remove file) inside _delta_log. Databricks reads the logs sequentially to build table state and generate snapshots. This makes operations like MERGE, DELETE, and UPDA TE safe even with concurrent writers in multi -team environments.

Q7. How do you ensure referential integrity between Silver and Gold tables?

You enforce referential integrity through DQ rules using join checks between dimensions a nd facts. Missing or mismatched dimension keys are routed to a reject table. This prevents corrupt Gold aggregates and ensures consistent metrics across finance and reporting use cases.

Q8. Why is merge -on-read faster than merge -on-write in some scenarios ?

Merge -on-read delays the cost of merging until read -time, reducing write -time latency. For ingestion -heavy use cases, this keeps Bronze/Silver ingestion fast while allowing compaction later. Cognizant often uses this approach when SLA requires fast inge stion but slower reporting windows.

Q9. What is checkpoint corruption and how do you prevent it?

Checkpoint corruption occurs when cluster issues, driver failures, or inconsistent filesystem states break the checkpoint directory. You prevent it by using stable ADLS Gen2, avoiding

manual edits, and enabling job retries with backoff policies. If corruption happens, you reset offsets using replay logic.

Q10. Why does Cognizant recommend Delta Live Tables (DLT) for standard ETL?

DLT gives declarative transformations, built -in DQ expectations, and full lineage tracking without manually writing checkpoints or streaming code. This makes ETL maintainable for large teams and reduces onboarding friction. It also minimizes coding mistakes since rules live alo ngside the pipeline.

Q11. How do you perform incremental merge with CDF in Databricks?

You enable delta.enableChangeDataFeed = true on source tables. Then, downstream jobs read table_changes() for only inserted/updated rows and apply MERGE into target. T his eliminates scanning the full table and is perfect for high -volume incremental pipelines.

Q12. Explain how you debug jobs with heavy memory usage.

You inspect Spark UI for skewed tasks, GC pressure, and executor metrics. You optimize by reducing shuff le, pruning columns early, and setting appropriate cluster memory settings. In worst cases, you repartition data or use broadcast joins only where safe.

Q13. How do you build a rollback mechanism in pipelines?

Since Delta supports time travel, rollback i s simply reading a previous version and overwriting current table. For complex systems, you maintain snapshot IDs for each batch in an audit table to enable fast restoration. This provides safety in high -risk deployments.

Q14. Why is cluster autoscaling i mportant?

Autoscaling dynamically adds executors when load increases and removes them when idle. This optimizes cost and guarantees SLA even during peak loads. Cognizant clients often have unpredictable workloads, making autoscaling essential.

Q15. How d o you schedule dependent jobs in Databricks?

You create Workflows where tasks depend on prior outputs, controlling sequence and failure handling. Workflows allow retries, email notifications, and branching logic (if/else). This replaces ADF dependence for lightweight orchestration.

Q16. Why do fact tables require surrogate keys?

Surrogate keys ensure stable joins even when business keys change over time. They also enhance query performance and improve compression. In Cognizant client environments, dimens ions sourced from multiple systems require harmonized surrogate keys.

Q17. How do you handle large Upsert operations efficiently?

You split MERGE into smaller batches using partition filters or CDF micro -batches. You also reduce shuffle by broadcasting small dimensions or coalescing partitions. Heavy MERGEs sometimes need cluster autoscaling and ZORDER maintenance.

Q18. What is watermarking and why is it needed?

Watermarking defines how long Spark should remember event -time state for streaming jobs. It protects state stores from infinite growth and prevents out -of-memory failures. This is critical for Cognizant workloads with late -arriving data from legacy sources.

Q19. Cognizant SQL – Get customers with increasing month -over-month purchase count

```
WITH m AS (
SELECT customer_id,
MONTH(order_date) AS mth,
COUNT(*) AS cnt,
LAG(COUNT(*)) OVER(PARTITION BY customer_id ORDER BY
MONTH(order_date)) AS prev
FROM orders
GROUP BY customer_id, MONTH(order_date)
```

)

SELECT * FROM m WHERE cnt > prev;

Q20. Cognizant PySpark – Fill missing values per group

from pyspark.sql import functions as F, Window

w = Window.partitionBy("customer_id").orderBy("date")

df_filled = df.withColumn(

"amount_filled",

F.last("amount", ignorenulls=True). over(w)

)

6. Accenture – Azure / Databricks Data Engineer

Q1. How do you make pipelines cloud -agnostic for multi -cloud clients?

You abstract storage and endpoints in configuration tables and use environment variables to reference cloud services. Your code never hardcodes S3/ADLS/GCP paths. Accenture uses this pattern heavily to reuse the same codebase across Azure, AWS, and GCP.

Q2. What is schema evolution and when should you NOT allow it?

Schema evolution allows new columns to appear in Bronze with out breaking ingestion. However, Accenture discourages automatic evolution in Silver and Gold because business logic might not understand new columns, breaking KPIs. Schema changes must be reviewed before propagation.

Q3. How do you create cost -optimized pipelines on Databricks?

Use job clusters instead of all -purpose clusters, enable autoscaling, keep tables compact, and prune unneeded columns early. Also schedule heavy jobs during off -peak hours with reserved capacity. Cost optimization is a key KPI for Accenture managed services.

Q4. What is a feature store and why is it useful?

Feature store centralizes computed features so ML and BI teams reuse standardized data. It ensures consistency between training and inference and avoids duplication of logic. Accenture often uses this to standardize credit scoring, churn prediction, and customer segmentation models.

Q5. Why are Bronze and Silver separated when clients want “clean data only ”?

Bronze stores the exact source data for audit and reprocessing. Silv er applies transformations, and missing Bronze would break traceability. Even if clients want clean outputs, separating zones prevents future debugging disasters.

Q6. How do you handle data pipelines requiring both batch and real -time?

You build streamin g ingestion for low -latency needs and a batch pipeline for historical backfills. Both feed into Bronze, then merge into Silver via MERGE logic or CDF. Accenture uses this hybrid design especially in telecom and BFSI projects.

Q7. How do you detect and fix clustering skew?

Profile column cardinality and use histograms to identify skewed distributions. Fix using salting, repartitioning, or enabling AQE ’s skew join optimization. Accenture also uses monitoring dashboards to detect skew trends early.

Q8. Why is horizontal scaling preferred for Spark?

Horizontal scaling increases node count instead of CPU on a single node, letting Spark parallelize tasks across many executors. Vertical scaling leads to memory pressure and garbage collection issues. Spark was d esigned for horizontal distributed computing.

Q9. How do you guarantee data lineage for compliance -heavy clients?

Unity Catalog captures table -level and column -level lineage automatically. For deeper lineage, you integrate it with Purview or Collibra. Li neage ensures auditability and reduces compliance risk.

Q10. How do you manage multi -environment deployments (Dev/Test/Prod)?

You keep separate storages, catalogs, and cluster policies. CI/CD pipelines deploy notebooks, configs, and workflows via Git. Se crets and keys vary per environment but code remains identical.

Q11. What is an incremental model and why use it?

Incremental models recompute only new or changed data instead of refreshing full tables. This dramatically reduces compute cost and prevents SLAs from breaking. It ’s essential in Accenture ’s large -scale telecom and retail projects.

Q12. How do you implement Slowly Changing Dimensions Type -1 and Type -2?

Type -1 overwrites old records, typically used for non -historical attributes. Type -2 create s historical versions with effective date ranges and current flags. Delta MERGE makes both easy to implement with consistent logic.

Q13. Why are surrogate keys useful in Gold tables?

They simplify joins, reduce storage consumption due to reduced index si ze, and avoid leaking sensitive information. Gold -level aggregates typically replace natural keys with surrogate keys for performance.

Q14. Explain the role of Optimize Writes.

Optimize Writes controls how Delta writes data by coalescing small output fil es during write operations. It reduces fragmentation and improves query performance automatically without manual compaction.

Q15. How would you design job retry strategies?

Implement exponential backoff, retry -once logic for flaky upstream failures, and dead -letter handling. Accenture pipelines integrate failure metrics into monitoring dashboards for proactive alerting.

Q16. What is lineage drift and how to handle it?

Lineage drift occurs when transformations stop matching metadata or governance documen tation, often due to silent code changes. Fix by enforcing version -controlled ETL, review PRs, and enabling Unity Catalog lineage checks.

Q17. Why use Azure Key Vault with Databricks?

Key Vault stores secrets securely so they aren ’t exposed in notebooks or logs. Databricks integrates directly, making credential management safer for multi -developer teams.

Q18. What is COGS optimization in cloud analytics?

COGS = Cost of Goods Sold. It refers to reducing cloud spending by minimizing compute and storage ov erhead. Accenture tracks COGS KPI and tunes pipelines to reduce unnecessary reads, writes, and long cluster runtimes.

Q19. Accenture SQL – Top 10 Products with Highest Revenue

```
SELECT product_id,
SUM(price * quantity) AS revenue
FROM sales
GROUP BY product_id
ORDER BY revenue DESC
LIMIT 10;
```

Q20. Accenture PySpark – Add Week Number and Fiscal Year

```
from pyspark.sql import functions as F
df_cal = df.withColumn("week_num", F.weekofyear("date")) \
.withColumn("fiscal_year", F.year(F.add_ months("date", 3)))
```

7. EY (Ernst & Young) – AZURE Data Engineer

Q1. Why are audit logs mandatory in EY deployments?

EY deals with regulatory clients where every data movement must be traceable. Audit logs record batch_id, row counts, timestamps, source file names, and transformation status. This ensures compliance and helps investigations during audits.

Q2. What is the biggest risk in schema drift?

A new column or datatype mismatch can break transformations, merges, and business KPIs. Silent drift lead s to corrupted dashboards and unresolved anomalies. EY mandates schema validation at Bronze/Silver boundaries to catch drift early.

Q3. How do you secure Delta tables for financial clients?

Restrict raw access to a handful of admins and expose only views with masked/sanitized data to analysts. Using Unity Catalog, enable row -level and column -level security. All sensitive fields can be tokenized or hashed in Silver/Gold.

Q4. How do you validate completeness of ingested data?

You compare expected file cou nts or expected row counts from source manifest tables. EY loads metadata into control tables and validates them during ingest. Incomplete batches are quarantined and alerts triggered automatically.

Q5. Why does EY emphasize strong referential integrity?

Incorrect joins or missing dimension keys cause inconsistent reporting across divisions. EY enforces key checks in Silver, and unmatched keys are sent to an exceptions table. This ensures that Gold tables are analytically correct.

Q6. How do you standard ize financial quantities?

Apply FX normalization, define standard units, round amounts with consistent rules, and maintain reference tables. EY ensures financial calculations use approved formulas to avoid discrepancies across departments.

Q7. What is the purpose of maintaining snapshot tables?

Snapshot tables capture state at daily/monthly intervals for audit and reporting. They are critical for EY clients needing historical financial views independent of SCD logic. They simplify re -running reports fo r prior periods.

Q8. Why use MERGE for audit updates instead of overwrites?

Overwrite operations destroy history; MERGE preserves incremental states and maintains audit consistency. MERGE also supports conditional updates and inserts, enabling precise handling of exceptions or corrections.

Q9. How does Databricks SQL help EY clients?

It provides a warehouse -style experience with strong governance while keeping compute scalable. Non -engineers can query data without touching Spark notebooks. It integrates cleanly with Power BI and Tableau dashboards.

Q10. What are control tables and why are they needed?

Control tables track pipeline metadata: batch status, expected counts, checkpoints, and SLA metrics. EY uses them to coordinate batch dependencies and pr event accidental reprocessing.

Q11. What causes double -counting in financial reports?

Improper deduplication, incorrect joins, or missing unique keys cause inflated aggregates. EY enforces strong dedup logic at Silver and proper natural key identificatio n in dimensions.

Q12. How do you design revenue fact tables at EY?

You use atomic granularity (transaction -level) with surrogate keys for dimensions. You normalize currencies, incorporate discounts, adjust taxes, and maintain audit fields. This guarantee s accuracy and reconciliation capability.

Q13. How do you orchestrate EY financial pipelines?

ADF triggers ingestion, then Databricks Workflows run Silver and Gold. Control tables track all steps, and failure notifications go to audit teams. Purview prov ides lineage for regulatory

checks.

Q14. Why avoid over -partitioning?

Too many small partitions slow down query planning and increase file open operations. EY recommends monthly partitions instead of daily for high -frequency data. Optimal partition sizes reduce cost and improve performance.

Q15. How do you manage GDPR deletions for sensitive clients?

Identify keys to delete, execute Delta DELETE, then run VACUUM after retention period. Maintain a deletion log table proving that records were removed. Thi s is mandatory for compliance.

Q16. Why use row -level security in Unity Catalog?

Some users are allowed to see only regional or division -specific records. Unity Catalog enforces RLS transparently, ensuring users see only allowed slices without duplicatin g tables.

Q17. How do you ensure deterministic surrogate keys?

Use hash -based keys with standardized concatenation of business attributes. Avoid sequences because parallel ingestion leads to inconsistent keys across environments. EY standardizes hashing formulas across pipelines.

Q18. What is a financial reconciliation layer?

It compares source totals vs target totals (e.g., sum of amounts per day). Any mismatch triggers alerts and quarantines. EY relies heavily on reconciliation due to strict audit requirements.

Q19. EY SQL – Month -to-Month Revenue Change

```
SELECT month,
revenue,
LAG(revenue) OVER(ORDER BY month) AS prev_month_rev,
```

revenue - LAG(revenue) OVER(ORDER BY month) AS diff

FROM monthly_revenue;

Problem Statement

You are given a transaction dataset with fields:

txn_id (may not be unique – upstream system sometimes duplicates)

account_id

txn_ts

amount

currency

source_system

financial audit pipelines, transactions must be deduplicated using business keys, not system

keys.

A duplicate transaction is defined as rows that have same:

(account_id, txn_ts, amount, currency)

Across multiple source systems, duplicates may arrive with different txn_id, so you cannot

use it.

Requirement:

Identify duplicates

Keep only the earliest row (by ingestion order or by txn_id)

Produce a final deduped table + exception table containing all duplicate records

```
from pyspark.sql import functions as F, Window

# Define composite business key window

w = Window.partitionBy(

"account_id",
```

```
"txn_ts",
"amount",
"currency"
).orderBy("txn_id") # earliest txn_id wins
# Rank duplicates
df_ranked = (df
.withColumn("dup_rank", F.row_number().over(w))
)
# Keep only master records (dup_rank = 1)
df_clean = df_ranked.filter(F.col("dup_rank") == 1) \
.drop("dup_rank")
# Capture duplicate exceptions
df_duplicates = df_ranked.filter(F.col("dup_rank") > 1)
```

8. Capgemini – Azure Data Engineer

Q1. Why does Capgemini rely heavily on reus able pipeline frameworks?

Global delivery centers handle many clients with similar ingestion needs. Reusable frameworks reduce development time and enforce consistent quality. They also simplify onboarding new team members.

Q2. What is the role of configuration -driven ETL?

Configuration -driven ETL allows file paths, schema mappings, validation rules, and load types to reside in metadata tables. Pipelines read config dynamically, minimizing code duplication. This creates scalable delivery models acr oss multiple clients.

Q3. Why are surrogate keys mandatory in dimensional modeling?

Natural keys vary across clients and systems; surrogate keys provide stable identity. They

also reduce join sizes and improve warehouse performance. Capgemini emphasizes global modeling standards using surrogate keys.

Q4. How do you choose cluster types for heavy workloads?

Use worker type optimized for memory if data skew exists or heavy joins occur. Use compute -optimized workers for high parallelism workloads. Capgemin i balances cost and SLA before finalizing cluster types.

Q5. What is the purpose of enforcing naming conventions?

Consistent naming helps governance, tooling integration, and multi -team collaboration. Capgemini uses naming standards for tables, schemas, and notebooks so environments look uniform and easier to navigate.

Q6. What is schema -on-read and why is it beneficial?

Schema -on-read keeps raw data flexible and avoids strict schema enforcement. This allows ingesting semi -structured sources like JSON w ithout preprocessing. Capgemini uses schema -on-read in Bronze to reduce ingestion failures.

Q7. Why does Capgemini encourage incremental ingestion whenever possible?

Full loads waste compute and extend batch windows. Incremental loads powered by CDC or modified timestamp reduce processing volume dramatically. It improves SLA consistency and reduces storage cost.

Q8. How do you handle GDPR for multinational clients?

Use tokenization, hashing, and regional data separation. Sensitive attributes must be mas ked, secured with column -level permissions, and logged for deletion audits. Capgemini ensures compliance with EU, UK, and APAC laws.

Q9. Why use small lookup tables in broadcast joins?

Broadcast avoids shuffle by sending small dimension tables to all executors. This drastically

accelerates joins between large facts and small dims. Capgemini pipelines rely on this pattern heavily for performance.

Q10. What is the importance of control -flow vs data -flow separation?

Control -flow involves orchestration ( dependencies, retries), while data -flow is actual transformations. Splitting them reduces complexity. Capgemini typically uses ADF for control and Databricks notebooks for data -flow.

Q11. How do you validate file -level structure before ingestion?

Define expected schema and optional validation checks — number of columns, headers, delimiter. Reject mismatched files into quarantine folders. This prevents ingestion of corrupted data.

Q12. How do you handle multi -file ingestion performance issues?

Use Auto L oader for incremental detection and parallel reading. Avoid excessive partitions and tune cloudFiles.maxFilesPerTrigger. Capgemini adjusts cluster sizes based on expected ingestion volume.

Q13. Why does Capgemini prefer star schemas over flat tables?

Star schemas provide normalized dimensions and simplified analytical facts. They reduce storage duplication and improve consistency across BI reports. Flat tables hide relationships and create inconsistent metrics.

Q14. How to ensure proper SCD2 implementati on in Capgemini projects?

You first close current record by updating is_current=false and set end timestamp. Then insert the new version with is_current=true. This preserves historical accuracy, crucial for audit -heavy domains.

Q15. Why is data enrichmen t done in Silver instead of Gold?

Silver ensures all conforming rules, validation, and enrichment are standardized before

analytics. Gold focuses solely on business -level aggregations. Mixing them complicates governance and reduces maintainability.

Q16. What is the danger of excessive repartitioning?

Unnecessary repartition operations trigger expensive shuffles and memory pressure. This increases compute cost and slows jobs. Capgemini pipelines repartition only when distribution imbalance is proven.

Q17. What is a slowly changing dimension and why is it needed?

SCD tracks changes in dimension attributes over time. It helps business users understand historical states rather than only the latest version. Capgemini applies SCD in finance, retail, health care, and SAP -driven analytics.

Q18. Why is column pruning important?

Column pruning reduces memory use and speeds up transformations by discarding irrelevant fields early. It prevents carrying 400 irrelevant JSON fields through entire pipelines.

Q19. C apgemini SQL – Identify duplicate phone numbers

```
SELECT phone, COUNT(*)
FROM customers
GROUP BY phone
HA VING COUNT(*) > 1;
```

Q20. Capgemini PySpark – Detect Columns with >50% Null Values

```
from pyspark.sql import functions as F
total = df.count()
df_nulls = df.select([ (F.count(F.when(F.col(c).isNull(), c))/total).alias(c) for c in df.columns
])
```

9. EPAM Systems — Azure Data Engineer

Q1. How would you design a reusable ingestion framework for multiple EPAM clients on Azure?

You build a metadata -driven ingestion layer where each source system is defined in a control table (path, format, schema, load type).

A single generic notebook reads this metadata and executes ingestion logic dynamically (Auto Loader / batch read) instead of hardcoding.

This design s cales across dozens of clients and reduces maintenance when sources/schema evolve.

Q2. Scenario: Source keeps adding new columns without notice. How do you avoid breaking downstream?

At Bronze, you allow schema evolution using Auto Loader ’s addNewColumns to avoid ingestion failures.

You maintain a “contract schema ” table for Silver and only map approved columns forward; all extra columns are parked or flagged.

You add alerts whenever new columns appear so architects decide whether to surface them to busin ess.

Q3. How do you handle different latency SLAs for different consumers (real -time vs daily batch)?

You maintain multiple Gold tables or views: low -latency aggregates built from streaming Silver, and more accurate daily aggregates built from batch Silv er.

Both read from the same curated Silver to avoid logic drift.

Each consumer (dashboards, APIs, ML) chooses the Gold flavor that matches its freshness vs accuracy trade -off.

Q4. Why is Auto Loader better than naive directory scanning for large lakes?

Directory scanning (LIST on big folders) becomes slower and more expensive as file count explodes.

Auto Loader uses notification or incremental listing with checkpoints, detecting only new files, and infers schema incrementally.

That makes it scalable to mi llions of files, which is standard in EPAM -scale environments.

Q5. How do you structure code so that the same ETL logic works in Dev, QA, and Prod?

No environment -specific values should be hardcoded in notebooks.

You externalize all configuration (paths, databases, secrets, concurrency limits) into environment -specific config tables or key vault.

The same codebase reads config based on env parameter and behaves correctly in all environments.

Q6. Scenario: A Silver join suddenly starts blowing up into hug e row counts. What do you check?

First verify the join condition; a missing key predicate or cross join will explode row count.

Then check for duplicate keys on the “dimension ” side that should be unique but aren ’t.

Finally, confirm that schema evolution didn’t change data types causing the join to misbehave (e.g., string vs int).

Q7. Why is it dangerous to blindly trust schema inference?

Inference depends on the sample files and may guess types incorrectly (string instead of numeric, boolean instead of string).

When more diverse data arrives, parsing errors or silent truncation happens.

EPAM -style robust pipelines define explicit schemas in critical areas and only use inference at Bronze with tightening at Silver.

Q8. How would you debug a long -running MERGE INTO job?

Inspect which step is heavy: scanning target, scanning source, or writing.

Check partitioning: if target is unpartitioned or not partitioned on the merge predicate, full scans happen.

Analyze skew on join keys and consider partitioning + Z ORDER, broadcasting small side, or splitting MERGE into smaller date chunks.

Q9. How do you design auditability so every row in Gold can be traced back to source file?

Each layer (Bronze → Silver → Gold) carries batch_id, source_system, and lineage_id or source_file columns.

Transformations preserve or derive a consistent lineage identifier through joins and aggregations.

You keep run -log tables linking batches to job runs, code commits, and data snapshots for full traceability.

Q10. Why is CDF (Change D ata Feed) powerful for EPAM -type projects?

CDF lets you query only changed rows (insert/update/delete) for a Delta table between two versions.

Instead of scanning a 2 TB Silver table to update Gold, you MERGE only the delta slice from CDF.

This is huge fo r SLA and cost, especially when many downstream tables depend on a single high-volume source.

Q11. Scenario: Your streaming job fails after a schema change and won ’t restart. What ’s your approach?

First check the checkpoint and table schema mismatch; the checkpoint may reference old schema.

Option 1: write a migration step to align schema and keep checkpoint.

Option 2 (last resort): archive existing checkpoint and restart job with a new checkpoint plus reprocessing logic to avoid duplicates.

Q12. How do you ensure idempotency when re -running batch jobs?

Use business keys + batch_id and MERGE instead of blind append.

The job checks whether a given batch has already been applied and either skips or updates accordingly.

You also design deduplication logic b ased on stable IDs, not on file arrival.

Q13. Why do you sometimes decouple ingestion clusters from heavy transformation clusters?

Ingestion is I/O -heavy but relatively simple; transformations are CPU/memory -heavy.

Separating them allows you to tune clus ter sizes per workload and scale them independently.

This also isolates noisy workloads and improves reliability.

Q14. How do you implement multi -region DR for critical Delta tables?

You use ADLS GRS or geo -replication to mirror data across regions.

Delta logs and table structure are replicated, and you maintain secondary Databricks workspace pointing to replica.

Failover policies decide when to switch jobs and BI to the secondary region.

Q15. What is the risk of frequent OPTIMIZE on very large tables?

OPTIMIZE is expensive: it rewrites files and can create I/O spikes, competing with production workloads.

Over -optimizing (e.g., every hour) burns compute budget and may even slow things down.

You schedule OPTIMIZE weekly/daily on hot partitions and leave c old ones untouched.

Q16. Scenario: You need to support both column -level and row -level security. How do you structure it?

You define views in Unity Catalog that apply column masking and row filters based on

current user/group.

Base tables remain fully detailed but only privileged groups can read them directly.

All BI/ML consumption goes through these secure views, not raw tables.

Q17. How do you measure pipeline “quality ” beyond just success/failure?

Track metrics like data freshness, DQ pass rate, SL A adherence, row -count deltas, and query performance trends.

Build dashboards consumed by both engineering and business owners.

This moves the conversation from “job is green ” to “is data actually trustworthy and on time? ”.

Q18. Why should you avoid too m any nested complex types (struct/array) in Gold?

Nested structures are good for raw and ML -friendly Silver, but BI tools struggle with them.

Flattening key parts in Gold makes it easier for analysts and improves SQL readability.

You keep nested fields onl y where they are truly part of the domain (e.g., JSON configs).

Q19. EPAM SQL Problem — Rolling 3 -Day Active Users With Gap Handling

Question:

You have a table events(user_id, event_date). Find users who were active on any 3 consecutive calendar days, where missing days (no events) break the streak.

```
WITH user_days AS (
SELECT DISTINCT user_id, event_date
FROM events
),
u AS (
SELECT
user_id,
```

event_date,

ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_date ) AS rn

FROM user_days

),

groups AS (

SELECT

user_id,

event_date,

DA TEADD(day, -rn, event_date) AS grp_key

FROM u

)

SELECT DISTINCT user_id

FROM groups

GROUP BY user_id, grp_key

HA VING COUNT(*) >= 3;

This uses the “date - row_number ” trick to detect contiguous streaks without gaps and returns

users who have at least one streak ≥ 3 days.

Q20. EPAM PySpark Problem — Sessionization With 30 -Minute Timeout

Question:

Given df(user_id, ts) sorted by ts, create a session_id where a new session starts if the gap

between events for same user is > 30 minutes.

from pyspark.sql import functions as F, Window

w = Window.partitionBy("user_id").orderBy("ts")

df_with_gap = (df

```
.withColumn("prev_ts", F.lag("ts").over(w))
.withColumn(
"new_session_flag",
F.when(
(F.col("prev_ts").isNull()) |
((F.col("ts").cast("long") - F.col("prev_ts").cast("long")) > 30 * 60),
1
).otherwise(0)
)
.withColumn("session_seq", F.sum("new_session_flag").over(w))
.withColumn("session_id", F.concat_ws("_", F.col("user_id"), F.col("session_seq")))
.drop("prev_ts", "new_session_flag", "session_seq")
)
```

This uses a cumulative sum over a flag to mark new sessions and generate deterministic session_id per user.

10. Citi — AZURE Data Engineer

Q1. How do you design a pipeline for daily risk exposure aggregation at a bank?

You ingest trades, positions, and market data into Bronze from multiple systems (FO/MO/BO).

Silver standar dizes identifiers, enriches with reference data, and converts all amounts to a base currency.

Gold aggregates exposures by counterparty, region, product, and hierarchy (desk → book → legal entity) with SCD2 dimensions.

Q2. Scenario: Some trades arrive lat e due to upstream delays. How do you prevent incorrect

end-of-day risk reports?

You define a cut -off time and treat anything after that as late data.

Late trades are still ingested but flagged and rolled into next -day adjustments or specific "T+n correction " tables.

If regulation demands exact view, you may recompute prior -day risk off -hours based on late input but mark versions clearly.

Q3. How do you enforce data retention for trading data?

You separate regulatory retention (e.g., 7+ years) fr om performance -retained working sets.

Raw Bronze may be archived to cheaper tiers while Silver/Gold keep only last N years for frequent queries.

Metadata in control tables states retention rules so jobs can lifecycle data automatically.

Q4. Why is Delta t ime travel important in banking?

It lets you reconstruct the exact view of a table at any past point, matching what a user or report saw.

This is essential for explaining discrepancies, audits, and regulatory reconstructions.

Citi-level environments often require "as-of" reporting for specific historical dates.

Q5. How do you design a reconciliation process between source system and data lake?

You compute aggregate metrics (count, sum(amount), sum(notional)) on both sides and log them into a reconciliati on table.

Any mismatch above threshold raises an alert and may block Gold refreshes.

Detailed exceptions can be investigated via join -on-key diff analysis.

Q6. Scenario: Trade -level P&L doesn 't sum up to desk -level P&L in the lake. What checks do you run?

Verify inclusion/exclusion rules in aggregations (e.g., cancelled trades).

Check currency conversion consistency and FX rates used in different layers.

Ensure SCD2 dimension keys haven ’t shifted due to incorrect effective dating.

Q7. How do you handle r isk of double -ingestion of the same file/batch?

You include a unique batch_id or file_hash in Bronze and enforce a uniqueness constraint at ingestion.

Any attempt to ingest same batch twice is either rejected or processed in idempotent fashion.

Downstream MERGE logic ensures duplicates do not create multiple valid rows.

Q8. Why is encryption at rest and in transit mandatory for Citi?

Banking data is highly sensitive; regulations require strong protection for PII, trades, and account info.

Encryption at r est (ADLS + CMK) and TLS in transit ensure that raw data cannot be trivially exfiltrated.

Security reviews verify keys are rotated and access logs are monitored.

Q9. How do you handle GDPR/CCPA ‘right to be forgotten ’ in a Delta -based warehouse?

You main tain a mapping of subject identifiers and run targeted DELETE operations on all impacted Delta tables.

After the retention period, VACUUM cleans up physical files.

You log deletions to a dedicated audit table showing compliance with erasure requests.

Q10. Why do you prefer MERGE to naive “overwrite partition ” in regulatory data?

Overwrite partition is simple but can wipe out valid history if your logic has bugs.

MERGE gives conditional control: only specific rows are inserted/updated/deleted.

In regulated environments, fine control wins over simplistic performance hacks.

Q11. How do you implement row -level access so that one region can ’t see another ’s data?

Use Unity Catalog row filters based on user attributes (like region) attached to AAD groups.

All B I tools must query through views that apply these filters instead of base tables.

This allows global deployment while protecting sensitive region -specific portfolios.

Q12. Scenario: You find a bug in the interest accrual logic that impacted last 6 months of data. How do you correct it?

You don ’t patch Gold only; you adjust logic where it belongs (Silver or a Gold business layer).

Then you reprocess back from the earliest impacted date, possibly using backfill jobs with date filters.

Each rerun gets its ow n batch id and version marker, so you can compare old vs new if needed.

Q13. How do you ensure that FX conversion is consistent across pipelines?

You centralize FX rates into a governed reference table with effective dates.

All Silver and Gold logic uses JOINs against that table instead of local hardcoded rates.

Tests validate that for a given date and currency pair, only one FX rate is valid.

Q14. What is dual -control change management and why does Citi need it?

Dual control means any critical change ( schema, transformation logic, retention policy) must be approved by at least two roles (e.g., dev + lead, or dev + risk).

This reduces risk of accidental or malicious changes that could affect financial reporting.

In practice, it maps to enforced PR review s and sign -offs.

Q15. Why are snapshot tables useful for regulatory reporting?

They store point -in-time views of positions/exposures, independent of SCD state changes later.

This simplifies reconstructing quarter -end or year -end reports when underlying d imensions change after the fact.

Snapshots are typically partitioned by snapshot_date.

Q16. How do you model SCD2 for counterparty dimensions?

You store counterparty_id, attributes (rating, sector, LEI, etc.), effective_from, effective_to, and is_current .

Trades link to surrogate key based on trade date falling inside the effective period.

This allows risk reports to reflect the correct rating valid on the trade or reporting date.

Q17. What is a “golden source ” in banking reference data?

It’s the designated authoritative system for a given data domain (e.g., counterparty, instrument).

When multiple sources conflict, golden source wins.

Your data lake respects those rules by overriding weaker sources in Silver.

Q18. Why is ACID so important for int raday risk pipelines?

You can ’t have partial updates where some trades reflect new data and others don ’t.

ACID ensures that each risk batch either fully commits or not at all, so risk numbers are consistent.

Rolling back an invalid run is also safe thanks to Delta ’s versioning.

Q19. Citi SQL Problem — Latest Balance + Overdraft Detection by Window

Question:

Given transactions(account_id, txn_ts, amount) with positive = deposit, negative = withdrawal. Compute each account ’s latest balance and flag wheth er it has ever gone below zero at any point in history.

```
WITH ordered AS (
SELECT
account_id,
```

txn_ts,

amount,

SUM(amount) OVER (

PARTITION BY account_id

ORDER BY txn_ts

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

) AS running_balance

FROM transactions

),

latest AS (

SELECT DISTINCT

account_id,

FIRST_VALUE(running_balance) OVER (

PARTITION BY account_id

ORDER BY txn_ts DESC

) AS latest_balance,

MAX(CASE WHEN running_balance < 0 THEN 1 ELSE 0 END)

OVER (PARTITION BY account_id) AS ever_overdraft

FROM ordered

)

SELECT DISTINCT account_id, latest_balance, ever_overdraft

FROM latest;

This uses a cumulative sum for running balance and then window functions to find fina l

balance + historically negative periods.

Q20. Citi PySpark Problem — Compute Liquidity Ladder (Cumulative in Time Buckets)

Question:

You have df(account_id, bucket_days, cashflow) where bucket_days ∈ {1,7,30,90,...}. Build a liquidity ladder per acco unt where each bucket shows cumulative cashflow up to that horizon.

```
from pyspark.sql import functions as F, Window
# assume bucket_days is numeric: 1, 7, 30, ...
w = Window.partitionBy("account_id").orderBy("bucket_days") \
.rowsBetween(Window. unboundedPreceding, Window.currentRow)
df_ladder = (df
.withColumn("cum_cashflow", F.sum("cashflow").over(w))
.orderBy("account_id", "bucket_days")
)
```

You can then pivot or expose this as a wide report. The key is cumulative window over ordered time buckets.

11. NTT DATA — Azure Data Engineer

Q1. Why is metadata -driven design almost mandatory for NTT DA TA -scale clients?

NTT DA TA serves many customers with similar patterns but slight variations in schema and frequency.

A metadata -driven framew ork lets you change behavior by updating config rows rather than branching code per client.

This massively improves maintainability and reusability across global engagements.

Q2. Scenario: Client insists on custom business logic per country while keeping one global pipeline. How do you design it?

You store country -level rules and thresholds in a rules table keyed by country + rule_type.

The ETL reads these rules at runtime and branches logic through parameterized transforms, not separate scripts.

You stil l keep core ingestion and validation common across all countries.

Q3. How do you ensure pipeline consistency across multiple regions and time zones?

Use UTC timestamps in storage layers and only convert to local time at presentation or final consumption.

Store time zone metadata with each record when needed.

Schedule jobs using UTC and treat daylight saving/time zone shifts explicitly in logic where required.

Q4. Why is SCD logic often centralized into shared libraries?

Multiple domains (customer, account, product) require SCD1/SCD2 behavior.

Duplicating MERGE logic everywhere is error -prone and makes bug fixes painful.

Centralized SCD utilities ensure consistent behavior, versioning, and testing.

Q5. How do you handle d ynamic schema mapping from multiple CRMs to a standard dimension?

You create a canonical schema and mapping table from source_field → canonical_field per source system.

Silver layer uses this metadata to transform source -specific columns into canonical na mes.

This allows you to plug in new CRMs without rewriting transformations.

Q6. Scenario: After rollout, job failures spike due to rare data patterns. How do you stabilize?

Log all failed records into DQ/exception tables with granular error codes.

Analyz e most frequent failure causes and adjust schema, casting, or business rules to handle them.

Introduce automated tests using synthetic edge -case data to stop regressions.

Q7. Why is state store size important in streaming jobs?

Large state store leads to high memory usage, slow checkpointing, and eventually OOM failures.

You must manage state with watermarks, proper key selection, and pruning logic.

NTT DA TA ensures streaming jobs have bounded state for long -term reliability.

Q8. How do you enforce tenan t isolation in a multi -tenant architecture?

Each tenant gets its own catalog/schema and sometimes its own ADLS container.

Access policies ensure no principal has rights across tenants unless explicitly intended (e.g., ops).

Code reads tenant_id parameter and uses tenant -specific paths and schemas.

Q9. What is the role of “run id ” in pipeline design?

A run id identifies one execution instance of a pipeline (often timestamp -based).

It allows you to trace logs, audit entries, and table updates associated with that run.

It’s essential for debugging, backfill, and reporting on operational metrics.

Q10. Why are validation -only runs useful?

Validation -only runs ingest files or rows into a staging area, apply DQ checks, but don ’t commit to final Silver/Gold.

Business and QA teams can review these results before “promoting ”.

This pattern reduces risk when onboarding new sources.

Q11. How do you reduce coupling between orchestration and transformation?

You keep transformation notebooks generic and parameterize d, and orchestration tools (ADF, Workflows) only pass parameters.

No orchestration -specific code should live inside notebooks.

This allows you to swap orchestrators without rewriting all ETL logic.

Q12. Scenario: Client wants data freshness SLA dashboards . How do you implement that?

Each job writes to a run_log table with start/end time, counts, and status.

Another job queries this table and summarizes freshness per dataset.

Dashboards visualize last successful batch time vs current time and flag if SLA t hresholds are breached.

Q13. Why use job clusters over all -purpose clusters in most production jobs?

Job clusters start clean every run, avoiding state leaks and environment drift.

They can be auto -sized per job and terminated immediately after completio n.

All-purpose clusters are more suited to ad -hoc development, not strict SLAs.

Q14. How do you test ETL logic safely before production?

Create synthetic input datasets covering edge cases and normal flows.

Run notebooks against Dev/QA environments with those inputs and assert expected outputs.

NTT DA TA often embeds these tests into CI/CD pipelines to block bad merges.

Q15. Why is backfill logic a separate concern from daily incremental logic?

Backfill often needs to handle older formats, huge volumes, and potential double -counting.

You may choose different clusters, schedule windows, and even specific code paths.

Keeping it separate prevents daily pipelines from becoming overly complex.

Q16. Scenario: You find that dags are too deeply chained and hard to reason about. How do you simplify?

Group related tasks into higher -level “units ” (e.g., Bronze pipeline, Silver pipeline, KPI pipeline).

Use fewer, coarser -grained orchestration nodes instead of hundreds of micro -tasks.

You still keep logical modularit y in code, just not in orchestration graph.

Q17. Why use Delta CDF instead of manually tracking high -water -marks?

High -water -mark logic per table is error -prone, especially during backfill or schema changes.

CDF inherently encodes what changed in Delta ’s log, and you can query changes between versions.

It simplifies incremental jobs and is easier for multiple downstream consumers to share.

Q18. What is an “idempotent sink ” and why is it critical?

An idempotent sink ensures writing the same batch twice d oesn’t create duplicate effects.

This is crucial when orchestrators or upstream systems occasionally retry failed jobs.

Delta MERGE -based sinks and dedup based on business keys are common patterns.

Q19. NTT DA TA SQL Problem — Latest Status Per Entity With History Filter

Question:

You have entity_status(entity_id, status, status_ts). Return, for each entity, the latest status in the last 7 days, but only for entities that were ever in status = 'ERROR' in that same 7 -day window.

```
WITH last_7 AS (
SELE CT *
FROM entity_status
WHERE status_ts >= DA TEADD(day, -7, CAST(GETDA TE() AS date))
),
has_error AS (
SELECT DISTINCT entity_id
FROM last_7
WHERE status = 'ERROR'
),
```

```
ranked AS (
SELECT
e.entity_id,
e.status,
e.status_ts,
ROW_NU MBER() OVER (
PARTITION BY e.entity_id
ORDER BY e.status_ts DESC
) AS rn
FROM last_7 e
JOIN has_error h ON e.entity_id = h.entity_id
)
SELECT entity_id, status, status_ts
FROM ranked
WHERE rn = 1;
```

This uses a filter window + conditional history and then pulls the latest status.

Q20. NTT DA TA PySpark Problem — Complex Pivot With Dynamic Columns

Question:

You have df(customer_id, metric_name, metric_value) where metric_name can be many dynamic values (e.g., credit_score, sal ary, age). Pivot into one row per customer with one column per metric.

```
from pyspark.sql import functions as F
metrics = [row.metric_name for row in df.select("metric_name").distinct().collect()]
df_pivot = (df
```

```
.groupBy("customer_id")
.pivot("met ric_name", metrics)
.agg(F.first("metric_value"))
)
```

This demonstrates dynamic pivoting. In production, you ’d avoid collect on huge domains, but for metrics (limited cardinality) this is realistic.

12. Altimetrik — Azure Data Engineer

Q1. How would you build a customer 360° pipeline for a digital product at Altimetrik?

You combine events from apps/web, CRM data, support tickets, and transactions.

Bronze ingests each source separately; Silver standardizes identities (email, phone, device IDs) and builds a unified customer key.

Gold surfaces a customer 360 table with behavioral, transactional, and engagement metrics for analytics and personalization.

Q2. Scenario: Business wants “hourly updated KPIs ” but doesn ’t want streaming complexity. What ’s your solution?

Use trigger -once Auto Loader or scheduled batch jobs every hour pulling only new files.

This retains batch semantics but gives near -real-time freshness.

You still use Delta + checkpoints to track what has already been processed.

Q3. How do y ou implement A/B test metric pipelines efficiently?

Treat experiment assignment and events as separate tables joined on user/session/time.

Gold aggregates compute metrics (conversion, retention, revenue) per variant, per segment.

You design the pipeline t o be parameterized by experiment_id and support both historical recompute and incremental updates.

Q4. Why is tracking event schemas (contracts) important in product analytics?

Frontends evolve fast: new properties are added, others change type.

If you d on’t track contracts, dashboards silently break or mix incompatible data.

Schema registry tables and contract testing ensure events follow documented structure.

Q5. How do you detect and handle bot or spam traffic in clickstream data?

You implement heuri stic and ML -based classifiers using features like request rate, pattern, geo, and user_agent.

Suspicious sessions are flagged in Silver and optionally excluded or marked in Gold metrics.

This keeps KPIs representing real user behavior rather than noise.

Q6. Scenario: A new product line launches; traffic doubles overnight. How do you prevent pipeline meltdown?

Autoscaling clusters, Auto Loader with proper backpressure, and optimized partitioning are critical.

You also pre -warm cluster pools and adjust shuffle partitions to match larger volume.

Dashboards may temporarily rely on coarser aggregations until full fidelity catches up.

Q7. Why is event -time processing more important than processing -time for user analytics?

User behavior must be grouped by w hen it actually happened, not when your system received it.

Late or retried requests could distort funnels or attribution if you rely on processing -time.

Event -time windows and watermarks keep metrics logically correct.

Q8. How do you version “metrics log ic” so old reports stay reproducible?

You store metric definitions in a table with version and effective_from timestamps.

Gold views use the appropriate metric version based on report date or explicit version flag.

This way, changes to metric definitions don’t retroactively rewrite historical business judgments unless explicitly desired.

Q9. How do you ensure near real -time monitoring of product KPIs?

You compute streaming or frequent micro -batch aggregations into short -term Gold (e.g., last 24 hours).

Alerts are based on moving averages and anomaly detection over that Gold.

Batch Gold tables provide more precise but slower historical metrics.

Q10. Why use Delta CDF to power downstream feature pipelines?

Feature pipelines don ’t need full table recompute ; they only care about changed events.

CDF allows you to generate incremental feature updates, training datasets, and real -time feature stores.

This is ideal for Altimetrik ’s product recommendation or personalization projects.

Q11. Scenario: Data scientists frequently request custom cuts of the data. How do you avoid ad-hoc chaos?

Provide a rich but governed semantic layer (Gold) with enough dimensions and metrics.

Teach them to build derived datasets on top of Gold instead of hitting raw tables.

If multiple DS teams need something repeatedly, promote it into the official Gold model.

Q12. How do you support multi -tenant SaaS analytics with shared infra?

Maintain a tenant_id column and policy -based row -level security in Unity Catalog.

Storage path s and table names might be shared, but queries are filtered to tenant ’s slice transparently.

Cluster policies can enforce limits per tenant to avoid noisy neighbors.

Q13. Why is sampling useful and dangerous at the same time?

Sampling can speed up explor ations and early -stage dashboards by reducing volume.

But if sampling is biased or misapplied, conclusions can be wrong (especially for low -frequency events).

You must document where sampling applies and never use it blindly in exec -facing metrics.

Q14. H ow do you handle evolving funnel definitions?

Funnel steps and conditions might change as product evolves.

You store funnel definitions in a config table and compute funnel metrics by reading the latest definition.

Historical funnels can be recomputed on demand using versioned definitions.

Q15. Why is aggregation push -down important?

Aggregating early (near Silver) reduces data volume flowing into Gold and BI.

It leverages cluster parallelism and reduces cost for interactive queries.

Altimetrik often pre -computes heavy metrics (retention, cohort analysis) instead of leaving everything to BI engines.

Q16. Scenario: A join between events and user table is too slow. What optimizations do you try?

Check if user table can be broadcast; if so, broadcast( user_dim).

Partition events on user_id to align distribution.

Ensure both sides use the same datatype and no unnecessary casts or expressions on join keys.

Q17. Why are wide tables sometimes preferred in ML -heavy workloads?

ML feature sets benefit from h aving many related features in a single row per entity/time.

This avoids expensive joins in each experiment and speeds up iteration.

In such cases, you trade 3NF purity for training performance.

Q18. How do you detect metric anomalies in trending dashboar ds?

Compute rolling windows (moving averages, std dev) and flag deviations beyond thresholds.

You can use time -series models or anomaly detection libraries, but even simple z -score alerts

can work.

Anomaly flags are stored along with metrics in Gold.

Q19. Altimetrik SQL Problem — First Purchase Attribution Window

Question:

Given events(user_id, event_ts, event_type, campaign_id) and orders(user_id, order_ts,

order_id), attribute each user ’s first order to the latest campaign event within 7 days before

the order.

```
WITH first_order AS (
SELECT
user_id,
MIN(order_ts) AS first_order_ts
FROM orders
GROUP BY user_id
),
candidate AS (
SELECT
fo.user_id,
fo.first_order_ts,
e.campaign_id,
e.event_ts,
ROW_NUMBER() OVER (
PARTITION BY fo.user_id
ORDER BY e.event_ts DESC
) AS rn
```

```
FROM first_order fo

JOIN events e

ON e.user_id = fo.user_id

AND e.event_ts <= fo.first_order_ts

AND e.event_ts >= DA TEADD(day, -7, fo.first_order_ts)

AND e.event_type = 'campaign_impression'

)

SELECT user_id, first_order_ts, campaign_id

FROM candidate

WHERE rn = 1;
```

This uses min order_ts + latest campaign event within window.

Q20. Altimetrik PySpark Problem — Build Dynamic Funnels

Question:

You have events( user_id, event_ts, event_type) and a funnel definition ["view_product",

"add_to_cart", "purchase"]. Detect which users completed the funnel in order within 2 days.

```
from pyspark.sql import functions as F, Window

# Assign step index for funnel

step_map = { "view_product": 1, "add_to_cart": 2, "purchase": 3 }

df_f = (df_events

.filter(F.col("event_type").isin(list(step_map.keys())))

.withColumn("step", F.lit(step_map)[F.col("event_type")])

)

w = Window.partitionBy("user_id").orderBy("event_ts")

df_seq = (df_f
```

```
.withColumn("next_step",
F.lag("step", -1).over(w)
)
.withColumn("start_ts",
F.first("event_ts").over(w.rowsBetween(Window.unboundedPreceding,
Window.currentRow))
)
)
# Simpler approach: aggregate per user
df_agg = (df_f
.groupBy("user_id")
.agg(
F.min("event_ts").alias("first_ts"),
F.max("event_ts").alias("last_ts"),
F.collect_list("step").alias("steps")
)
.withColumn("completed",
(F.array_sort("steps") == F.array ([F.lit(1), F.lit(2), F.lit(3)])) &
((F.col("last_ts").cast("long") - F.col("first_ts").cast("long")) <= 2 * 24 * 3600)
)
)
df_completed = df_agg.filter("completed = true")
```

13. Infosys — AZURE Data Engineer

Q1. How do you design an reusable ETL framework for multiple clients?

Create metadata -based ingestion templates where source format, schema, and file paths come from config tables.

Transformations should be handled using dynamic mapping logic that reads from metadata rather than hardcoded rules.

This allows onboarding new projects quickly by updating metadata entries, not rewriting pipelines.

Q2. Scenario: Client delivers CSV files with inconsistent date formats. How do you fix it?

Capture multiple allowed date patterns through a standar dization function in Bronze → Silver.

Log records that fail parsing into an error table for review instead of failing full pipeline.

Apply strict schema enforcement downstream so Silver data remains trustworthy.

Q3. Why use Auto Loader for enterprise inge stion?

Auto Loader uses file notifications to incrementally detect new files without scanning folders.

It avoids costly LIST operations on large ADLS directories.

Also supports schema inference, schema evolution, and efficient checkpointing.

Q4. How do you ensure smooth handover between onsite and offshore teams?

Use Git -based versioning for all notebooks and SQL scripts.

Maintain runbooks, validation documents, lineage diagrams, and SLA dashboards.

Ensure pipeline logs and audit tables are accessible t o the offshore team to resume failures.

Q5. What is the most efficient partitioning for TeraScale datasets?

Partition based on low -cardinality columns like date or region for optimal pruning.

Avoid over -partitioning, which creates thousands of small file s and hurts performance.

Monitor query patterns to reshape partitions as data volume grows.

Q6. Scenario: A Silver table suddenly grows 10× in size. What do you investigate?

Check whether deduplication logic broke or watermark logic failed, causing repro cessing.

Confirm if new source partitions were delivered unexpectedly.

Review MERGE predicates that may have created duplicate SCD2 rows.

Q7. Why implement business rules at Silver and not Bronze?

Bronze should store raw, unmodified data for traceability and replay.

Silver standardizes, validates, and applies initial business logic safely.

Gold handles full transformations and KPI logic after data is clean.

Q8. How do you manage large -scale Delta OPTIMIZE jobs?

Schedule OPTIMIZE during off -peak hours to avoid cluster overload.

Use OPTIMIZE table ZORDER BY (column) selectively for high -impact columns only.

Avoid optimizing partitions with low query frequency.

Q9. What is data observability and why is it important?

Observability ensures health of data pipelines via metrics like freshness, volume, drift, and lineage.

It helps proactively detect anomalies before they impact reports.

Infosys -style managed services rely on observability to meet SLA guarantees.

Q10. Scenario: Client has multiple environment s (Dev, QA, UA T, Prod). How do you sync schemas?

Use Unity Catalog schema enforcement and versioned Delta table definitions.

Promote schema through CI/CD pipelines tied to Git.

Implement pre -deployment validation scripts that compare schemas between envir onments.

Q11. How do you scale pipelines when volume jumps from 100GB to 10TB?

Increase worker nodes or move to autoscaling clusters to handle high shuffle load.

Repartition source data based on common filter columns.

Switch from JSON/CSV to Parquet or D elta formats for performance.

Q12. Scenario: Your pipeline runs but Power BI shows incomplete data for today's date. What is wrong?

Daily partitions may not be fully written or Gold aggregates might be using stale snapshots.

Auto Loader might have pendin g micro -batches not yet processed.

Fix by inspecting Silver completeness and forcing Gold regeneration for incomplete partitions.

Q13. Why introduce surrogate keys in enterprise data models?

Source system keys may not be stable or unique across all domai ns.

Surrogate keys unify different source systems under a common model.

They help maintain referential integrity even when upstream systems change.

Q14. How do you enforce PII governance across multiple projects?

Use column masking policies in Unity Catalog.

Control access via roles instead of direct table permissions.

Store sensitive fields encrypted or hashed in Silver and Gold.

Q15. How do you track SLA adherence?

Store pipeline start/end time, row count, and batch ID in a run log table.

Use Azur e Monitor or Log Analytics to trigger alerts on SLA breaches.

Create dashboards for business teams tracking pipeline timeliness.

Q16. Scenario: Your job takes 2 hours daily; business wants it under 30 minutes. Steps?

Analyze Spark UI for skew, spills, or redundant shuffles.

Optimize expensive joins using broadcast or bucketing.

Implement incremental logic using CDF instead of full -table scans.

Q17. Why is checkpoint cleanup important?

Old checkpoint state increases storage cost and slows streaming recov ery.

Periodic cleanup ensures the system only retains required state.

Reduces chances of corrupted or bloated checkpoints.

Q18. How to maintain dimension tables in SCD2?

Use MERGE to expire old records and insert updated versions.

Maintain valid_from and valid_to timestamps for historical tracking.

Ensure natural keys are stable; otherwise generate consistent surrogate keys.

Q19. SQL Problem -- Find all employees whose salary is more than 20% above their

department ’s average salary.

Return: emp_id, e mp_name, dept, salary, dept_avg_salary.

SELECT dept, COUNT(*)

FROM employees

GROUP BY dept

HA VING A VG(salary) > 80000;

Q20. PySpark Problem -- Write a PySpark transformation to find the maximum income for

each city and return a DataFrame with columns:

df2 = df.groupBy("city").agg(F.max("income").alias("max_income"))

14. TCS — AZURE Data Engineer

Q1. How do you design a fully reusable ingestion pipeline for multi -domain clients?

Use a standardized Bronze ingestion template driven by metadata from a co ntrol table.

Each domain configures source paths, schema, delimiter, and validation rules.

This enables rapid deployment across BFSI, Telecom, Retail, and Healthcare clients.

Q2. Scenario: Client data arrives 4 hours late every day. How do you maintain SL A?

Implement ADF triggers based on file arrival instead of fixed timers.

Use alerting to notify delay and track delay patterns.

Ensure downstream pipelines process data incrementally once files land.

Q3. Why implement Medallion Architecture at scale?

It cleanly separates raw, standardized, and business -curated data.

Allows independent reprocessing of Silver and Gold without breaking Bronze.

Greatly simplifies debugging and root cause analysis.

Q4. Role of Data Quality rules in BFSI clients of TCS

Rules validate account numbers, date formats, missing values, and numeric bounds.

DQ failures route records to an error quarantine table.

Detailed DQ logs help in reconciliation and audits.

Q5. How do you scale Auto Loader for millions of small files?

Use not ification mode instead of file scanning mode.

Enable Auto Optimize for small -file compaction.

Set efficient batch triggers (processingTime = “1m”).

Q6. Scenario: Gold table shows mismatched top -level KPIs. Root cause?

Silver transformations may be droppi ng late -arriving data.

Incorrect joins or missing lookup values may distort results.

Gold aggregates may be reading from outdated versions.

Q7. Why use Delta for audit -heavy industries like Telecom and BFSI?

Delta supports versioning, ACID, and time trav el.

Allows regulators to see historical snapshots.

Supports rollback without losing underlying data.

Q8. Explain CDC merge logic.

Use MERGE INTO on natural key with flags like I/U/D.

Update existing rows, insert new ones, and soft -delete deleted rows.

Ensure MERGE conditions are efficient using partition pruning.

Q9. How do you ensure consistent environments across Dev → QA → Prod?

Define schemas, tables, and permissions in Terraform or IaC templates.

Use GitOps for promoting notebooks and SQL scripts .

Compare schemas automatically using pre -deployment checks.

Q10. Why use bucketing?

Improves join performance by pre -hashing data into predictable buckets.

Reduces shuffle overhead during join execution.

Useful when joining massive fact tables repeatedl y.

Q11. Scenario: You see high shuffle spill warnings. What to do?

Increase cluster memory or adjust shuffle partitions.

Broadcast small tables and avoid unnecessary wide transformations.

Monitor skew patterns and address them via salting.

Q12. Why use CDF in large -scale banking transformations?

CDF emits only changed records from Silver to Gold, eliminating full table scans.

Ideal for updating risk aggregations and customer 360 views incrementally.

Reduces runtime significantly.

Q13. How do you handle historical data loads?

Load historical files into Bronze with batch_id tracking.

Process into Silver using date -based partitions.

Ensure incremental pipelines skip partitions already processed.

Q14. Why is indexing important for large dimensions?

ZORDE R improves locality of data on key columns.

Allows BI queries to scan only relevant files.

Critical for customer -level and SKU -level analysis.

Q15. Explain cluster pools in Databricks.

Cluster pools maintain pre -warmed nodes to reduce cluster startup tim e.

Used heavily when multiple jobs run frequently.

Improves job SLA and reduces wait time.

Q16. Scenario: Customer complains about stale Gold dashboards. Fix?

Inspect job runs for failures or missed triggers.

Validate Silver completeness and reprocess im pacted partitions.

Ensure BI tools refresh caches after Gold update.

Q17. How do you improve MERGE performance?

Partition on natural key or date.

Use WHEN MA TCHED AND filters to reduce unnecessary updates.

Optimize table with ZORDER on join key.

Q18. Why use structured streaming?

Processes data continuously with micro -batch guarantees.

Offers checkpointing, state management, and consistent writes.

Works well with EventHub and Auto Loader.

Q19. SQL Problem

SELECT customer_id, SUM(amount)

FROM transac tions

GROUP BY customer_id

HA VING SUM(amount) > 5000;

Q20. PySpark Problem

df2 = df.withColumn("year", F.year("order_date"))

15. HCLTech — AZURE Data Engineer

Q1. How does HCLTech structure large multi -year data modernization projects?

They use phased migration: Lift & Shift → Standardization → Optimization → AI/ML enablement.

Data moves into ADLS Bronze, refined into Silver, then migrated into curated Gold layers.

Each phase includes validation, governance, and automation.

Q2. Scenario: Multiple busi ness units use different formats for the same data. How to unify?

Enforce a unified Silver schema with strict datatypes.

Map source variations using metadata transformations.

Keep a harmonization table to track mapping between BU -specific fields and canonical fields.

Q3. Why use Delta for migration from legacy systems?

Delta supports partial loads, business -key merges, and versioning.

Allows easy rollback if migrated data fails validation.

Reduces time needed for reconciling legacy vs new environmen t.

Q4. How do you detect data drift in heterogeneous systems?

Compare value distributions using statistical methods like KS -test.

Monitor anomaly metrics and thresholds for each column.

Alert business teams before issues hit dashboards.

Q5. Scenario: Cu stomer data from SAP shows random duplicate rows. Fix?

Deduplicate using window functions on natural business keys.

Implement batch -level dedup in Silver layer.

Add validation steps to detect duplicate patterns upstream.

Q6. Why use DLT (Delta Live Table s)?

DLT enforces dependencies and data quality rules automatically.

Makes ETL pipelines declarative rather than hand -coded.

Ideal for HCLTech ’s delivery model with global teams.

Q7. How do you reduce ingestion costs?

Use Auto Loader notification mode instead of LIST -based scanning.

Compact small files early using Auto Optimize.

Use lifecycle rules to move old files to cool or archive storage.

Q8. Explain a multi -layer security design.

Use AAD groups → Unity Catalog permissions → Table -level masking → Tokenization in

Silver.

Audit logs capture access across all layers.

Least -privilege access ensures PII safety.

Q9. Scenario: Silver to Gold pipeline fails due to schema mismatch. Root cause?

Silver schema may have evolved while Gold expects older struc ture.

Fix by syncing schema metadata and adjusting transformation logic.

Add schema validation pre -check steps.

Q10. Why enforce surrogate keys?

They ensure consistent joins across domains, even when source keys differ.

Useful for building unified dimens ions like customer and vendor.

Prevent join errors due to type mismatch.

Q11. Why maintain reconciliation reports?

Ensures source → Bronze → Silver row counts match.

Detects missing or duplicated records early.

Critical for Finance and Healthcare clients .

Q12. Explain databricks cluster modes.

All-purpose clusters for development and interactive workloads.

Job clusters for scheduled pipelines.

High -concurrency clusters for BI workloads.

Q13. Scenario: Processing time for Gold aggregates increases daily. Fix?

Switch to incremental CDF -based aggregation.

Partition Gold tables based on date.

Optimize heavy partitions weekly.

Q14. Why prefer Parquet/Delta over CSV for mid -size clients?

Parquet compresses data and saves storage.

Delta adds ACID reli ability and better performance.

CSV is slow, large, and error -prone.

Q15. Explain row -level security implementation.

Use dynamic views that filter rows based on user attributes.

Map AAD groups to row -level rules.

Ensure policies apply consistently across layers.

Q16. Scenario: Data analyst complains about slow queries. Fix?

Create optimized Gold summary tables.

Enable ZORDER on critical columns.

Provision a SQL warehouse for BI querying.

Q17. How do you handle late -arriving CDC data?

Use watermarks and timestamps to capture late rows.

Apply MERGE -based correction into Silver and Gold.

Recompute impacted aggregates.

Q18. Why use Data Validation Frameworks?

They centralize rules, reduce code duplication, and enforce consistent quality.

Great for heal thcare and BFSI clients who require validated data.

Creates auditable DQ logs.

HCL SQL Problem – Detect Revenue Drop by Region (Month -on-Month)

You have a table:

sales(region, sale_month, revenue)

-- sale_month is DA TE or first day of the month

Requir ement:

For each region, find months where revenue dropped compared to the previous month.

Return: region, sale_month, revenue, prev_month_revenue, drop_amount.

Answer:

```
WITH sales_with_prev AS (
SELECT
region,
sale_month,
revenue,
LAG(revenue) OVER (
PARTITION BY region
ORDER BY sale_month
) AS prev_month_revenue
FROM sales
)
```

```
SELECT

region,

sale_month,

revenue,

prev_month_revenue,

(prev_month_revenue - revenue) AS drop_amount

FROM sales_with_prev

WHERE prev_month_revenue IS NOT NULL

AND revenue < prev_month_revenue;
```

This uses LAG + PARTITION BY region to compare each month with the previous month

and filter only where revenue dropped.

HCLTech – Flag High -Risk Customers by 3-Month Avg Spend

You have a DataFrame:

df(customer_id, month, spend)

# month = first day of that month (date or string)

Requirement:

For each customer_id, compute the 3 -month rolling average spend ordered by month.

Add a column is_high_risk = True if 3 -month average spend is less than 1000, else False.

```
from pyspark.sql import functions as F, Window

# Window: 3 rows (months) including current, per customer ordered by month

w = (Window

.partitionBy("customer_id" )

.orderBy("month")

.rowsBetween( -2, 0))
```

```
df_with_avg = (
df
.withColumn("avg_3_month_spend", F.avg("spend").over(w))
.withColumn(
"is_high_risk",
F.when(F.col("avg_3_month_spend") < 1000, F.lit(True))
.otherw ise(F.lit(False))
)
)
```

16. Wipro — AZURE Data Engineer

Q1. How does Wipro structure end -to-end Azure Data Engineering solutions?

They use ADF for orchestration, ADLS for storage, and Databricks for transformations.

Pipelines follow medallion layering to maintain clarity and quality.

Automation and governance are emphasized across environments.

Q2. Scenario: Client demands row -level lineage for regulatory reasons. How do you provide it?

Use Unity Catalog lineage to capture column -to-column transformations.

Store metadata snapshots of each Delta version for audit replays.

Expose lineage graphs through governance dashboards.

Q3. Why unify multiple data sources into canonical models?

Canonical models stand ardize attributes across systems.

Reduces downstream complexity and makes ML features cleaner.

Ensures consistent dimensions across dashboards.

Q4. How do you handle multi -tenant platforms?

Create separate UC catalogs per tenant.

Isolate ADLS containers, key vaults, compute clusters, and audit logs.

Apply tenant -level RBAC to ensure strong isolation.

Q5. Scenario: Gold layer queries degrade over time. What ’s wrong?

File sizes likely became suboptimal due to small writes.

ZORDER may not match new dashboa rd filter patterns.

Re-optimize Gold partitions and adjust ZORDER strategy.

Q6. Why use ADF + Databricks instead of ADF alone?

ADF handles orchestration, while Databricks handles compute -heavy jobs.

Databricks supports distributed processing, Delta Lake, and complex transformations.

ADF alone cannot efficiently process TB -scale data.

Q7. How to implement strong SLA monitoring?

Track per -step run logs stored in Delta tables.

Use Azure Monitor alerts for failures and delays.

Visualize SLAs in dashboards shared with leadership.

Q8. Explain the benefit of job clusters.

They spin up clean for every run, avoiding state contamination.

Cheaper than all -purpose clusters for scheduled jobs.

Enable optimized configurations for production loads.

Q9. Scenario: So urce fields change type unexpectedly. How do you handle this safely?

Validate schema before write and quarantine mismatching records.

Alert engineering teams immediately to adjust mapping configurations.

Never auto -coerce fields if data integrity is criti cal.

Q10. Why maintain data contracts?

Data contracts specify schema, freshness, SLAs, and DQ expectations.

Ensure upstream teams deliver data consistently.

Reduce unexpected load failures and schema breakages.

Q11–Q18 Continued

Q11. What is the role of reconcile tables?

They count rows, check duplicates, and compare source vs target metrics.

Helps ensure ingestion correctness and prevents data drift.

Mandatory for BFSI and Telecom clients.

Q12. Why implement business rules in Gold rather than Silver?

Gold is meant for BI logic and KPIs.

Silver retains standardized but raw values for reprocessing flexibility.

Separating these layers avoids breaking upstream dependencies.

Q13. Scenario: Pipeline slow because of a giant join. Steps?

Broadcast small tab les to avoid shuffle.

Enable AQE for dynamic optimization.

Repartition data based on join keys.

Q14. Why use surrogate keys even with clean natural keys?

Natural keys may change or not be unique across systems.

Surrogate keys provide stability in dimensi on tables.

Useful for maintaining consistent SCD2 history.

Q15. How to enforce PII masking?

Use dynamic views with masked fields for unauthorized users.

Encrypt or tokenize sensitive information in Silver.

Implement ABAC policies at UC level.

Q16. Scena rio: Delta table hits version history retention limit. Solution?

Enable checkpointing and archive older versions via snapshot exports.

Increase retention if business requires long audit history.

Vacuum older files cautiously.

Q17. Why use notebook modularization?

Breaks complex transformations into reusable components.

Reduces duplication and simplifies debugging.

Supports independent testing and promotion.

Q18. How do you handle low -latency streaming?

Use trigger intervals of 1–10 seconds based on workload.

Keep transformations lightweight and avoid excessive joins.

Use stateful operations only when necessary.

SQL Problem - You have a table:

logins(emp_id, login_date)

-- one row per employee per day they logged into the system

Requirement:

Find employees who logged in on at least 3 consecutive days.

Return distinct emp_id.

Answer:

```
WITH numbered AS (
SELECT
emp_id,
login_date,
ROW_NUMBER() OVER (
```

```
PARTITION BY emp_id
ORDER BY login_date
) AS rn
FROM logins
),
groups AS (
SELECT
emp_id,
login_date,
login_date - rn * INTERVAL '1 day' AS grp_key
FROM numbered
)
SELECT DISTINCT emp_id
FROM groups
GROUP BY emp_id, grp_key
HA VING COUNT(*) >= 3;
```

Pattern: date - row_number() groups consecutive days; any group with count ≥ 3 gives a 3+ day streak.

Wipro – Find Top Category per Customer by Spend

You have:

df(customer_id, category, amount)

Each row is a purchase.

Requirement: For each customer_id, find the category with the highest total amount spent.

Return DataFrame with columns: customer_id, top_category, total_spent_in_category.

```
from pyspark.sql import functions as F, Window

# First: aggregate spend per customer & category

df_agg = (

df.gr oupBy("customer_id", "category")

.agg(F.sum("amount").alias("total_spent"))

)

# Window: per customer, order categories by total_spent desc

w = Window.partitionBy("customer_id").orderBy(F.col("total_spent").desc())

df_ranked = (

df_agg

.with Column("rnk", F.row_number().over(w))

.filter(F.col("rnk") == 1)

.select(

"customer_id",

F.col("category").alias("top_category"),

F.col("total_spent").alias("total_spent_in_category")

)

)
```

■ PROJECT 1: Real -Time Clickst ream Ingestion (Event Hub → Auto Loader → Delta → KPIs)

1. What this project actually is (summary)

You're building a real -time analytics pipeline for an e -commerce website:

Events (page views, clicks, add -to-cart, purchases) land in Azure Event Hubs

Databricks Auto Loader consumes them into Bronze (raw) Delta

A streaming job cleans and enriches into Silver

Gold tables store hourly/daily KPIs for dashboards (Power BI / SQL Warehouse)

This is a textbook Azure + Databricks streaming + Delta Lake proje ct.

2. Tech stack

Azure Event Hubs

Azure Data Lake Storage Gen2 (ADLS)

Azure Databricks

Auto Loader

Structured Streaming

Delta Lake

Unity Catalog (optional but recommended)

Power BI (for reporting)

3. Data model

Raw clickstream event (from Event Hub / website):

{

"event_id": "uuid",

"user_id": "user123",

"session_id": "sess_456",

"event_type": "page_view | click | add_to_cart | purchase",

"page": "/product/iphone -16",

"referrer": "direct | google | facebook",

"timestamp": "2025 -11-23T10:15:30Z",

"device": "mobile | desktop",

"country": "IN"

}

4. Step -by-step implementation

Step 1 – Create Event Hub + ADLS + Databricks Workspace

Event Hub: clickstream -eh

ADLS container: raw, curated

Databricks workspace: attach to the same VN et if needed

You won ’t put this infra code in the ebook; just describe it.

Step 2 – Bronze ingestion with Auto Loader (from Event Hub checkpointed directory)

Assume Event Hub messages are being written to ADLS landing as JSON (via Event Hub

Capture or a simple consumer). Auto Loader reads from landing → Bronze.

```
from pyspark.sql import SparkSession

from pyspark.sql import functions as F

spark = SparkSession.builder.getOrCreate()

bronze_path = "abfss://raw@<storage_account>.dfs.core.windows.net/clickstr eam/bronze"

chk_bronze =

"abfss://raw@<storage_account>.dfs.core.windows.net/clickstream/_chk_bronze"

df_bronze = (spark.readStream

.format("cloudFiles")

.option("cloudFiles.format", "json")

.option("cloudFiles.schemaEvolutionMode", "addNewCo lumns")

.load(bronze_path)

)

(df_bronze.writeStream

.format("delta")
```

.option("checkpointLocation", chk_bronze)

.outputMode("append")

.trigger(processingTime="30 seconds")

.start("abfss://raw@<storage_account>.dfs.core.windows.net/c lickstream/bronze_table")

)

What you achieved:

Continuous ingestion, raw JSON stored as Delta with ACID + schema evolution.

Step 3 – Silver: cleaning, deduplication, standardization

Operations here:

Parse timestamp → event_ts

Hash user_id for privacy

Deduplicate by (session_id, event_ts, event_type)

from pyspark.sql import functions as F, Window

bronze_tbl = "clickstream_bronze"

df_bronze = spark.readStream.table(bronze_tbl)

df_silver = (df_bronze

.withColumn("event_ts", F.to_timestamp("times tamp"))

.withColumn("event_date", F.to_date("event_ts"))

.withColumn("user_hash", F.sha2(F.col("user_id").cast("string"), 256))

.drop("user_id")

.dropDuplicates(["session_id", "event_ts", "event_type"])

)

chk_silver = "abfss://raw@<storage _account>.dfs.core.windows.net/clickstream/_chk_silver"

(df_silver.writeStream

.format("delta")

.option("checkpointLocation", chk_silver)

.outputMode("append")

.partitionBy("event_date")

.start("abfss://curated@<storage_account >.dfs.core.windows.net/clickstream/silver_table")

)

Step 4 – Gold: hourly/daily KPIs

Examples:

Hourly page hits per page

Active users per hour

Event -type distribution (click, add_to_cart, purchase)

silver_tbl = "clickstream_silver"

df_silver = spark .readStream.table(silver_tbl)

from pyspark.sql import functions as F

df_gold_hourly = (df_silver

.groupBy(

F.window("event_ts", "1 hour").alias("event_window"),

"page"

)

.agg(

F.count("*").alias("hits"),

F.countDistinct("session_id").alias("unique_sessions"),

F.countDistinct("user_hash").alias("unique_users")

)

.withColumn("window_start", F.col("event_window").start)

.withColumn("window_end", F.col("event_window").end)

.drop("event_ window")

)

chk_gold =

"abfss://curated@<storage_account>.dfs.core.windows.net/clickstream/_chk_gold"

(df_gold_hourly.writeStream

.format("delta")

.option("checkpointLocation", chk_gold)

.outputMode("complete")

.start("abfss://curated@<sto rage_account>.dfs.core.windows.net/clickstream/gold_hourly_kp

is")

)

Step 5 – Optimizations

Enable Auto Optimize on silver/gold tables

Periodic OPTIMIZE ... ZORDER BY (event_date, page)

Use Databricks SQL Warehouse + Power BI DirectQuery

How to sell th is in interviews

You talk about:

Why you used Auto Loader instead of ADF for high -volume streaming

How you handled deduplication and late events

How Silver/Gold design supports both dashboards and future ML

■ PROJECT 2: Enterprise CDC + SCD Type -2 (Oracle/SAP → Delta Lake)

1. Summary

This project is about incremental ingestion from a transactional system (Oracle/SAP) and

maintaining a historical dimension using SCD Type -2 in Delta.

ADF pulls CDC (changed dat a) from source

Landed as Bronze (raw changes)

Databricks cleans to Silver

Gold is a full SCD2 dimension maintained via MERGE

2. Assumed CDC input

From Oracle log -based CDC or SAP:

customer_id | customer_name | phone | address | updated_at | op

---------- | ---------- | ------- | ---------- | ---------- | --

101 | Raj Kumar | 999... | Chennai | 2025 -11-23 10:00:00 | I

101 | Raj K | 999... | Chennai | 2025 -11-24 09:00:00 | U

102 | Kiran | 888... | Bangalore | 2025 -11-24 10:00:00 | I

op = I (insert), U (update), D (delete)

3. Bronze load

ADF → ADLS → Bronze table:

bronze_path = "abfss://raw@<storage>.dfs.core.windows.net/cdc/customer/bronze"

df_bronze = (spark.read

.format("csv")

.option("header", "true")

.load(bronze_path)

)

df_bronze.write.format("delta").mode("append").saveAsTable("cdc_customer_bronze")

4. Silver: type casting + DQ

```
from pyspark.sql import functions as F

df_bronz e = spark.table("cdc_customer_bronze")

df_silver = (df_bronze

.withColumn("updated_at", F.to_timestamp("updated_at"))

.withColumn("op", F.upper(F.col("op")))

.filter(F.col("customer_id").isNotNull())

)

df_silver.write.format("delta").mode ("overwrite").saveAsTable("cdc_customer_silver")
```

5. Gold: SCD Type -2 dimension

Gold table structure

```
CREA TE TABLE IF NOT EXISTS dim_customer_scd2 (

customer_id STRING,

customer_name STRING,

phone STRING,

address STRING,

start_ts TIMESTAMP,

end_ts TIMESTAMP,

is_current BOOLEAN

) USING delta;
```

SCD2 MERGE logic

```
from delta.tables import DeltaTable

from pyspark.sql import functions as F

silver_df = spark.table("cdc_customer_silver")

dim_path = "abfss://curated@<stor age>.dfs.core.windows.net/dim/dim_customer_scd2"
```

```
dim_delta = DeltaTable.forPath(spark, dim_path)

# 1. Close old records for updates

updates_df = silver_df.filter("op IN ('U', 'D')")

(dim_delta.alias("t")

.merge(

updates_df.alias("s"),

"t.customer_id = s.customer_id AND t.is_current = true"

)

.whenMatchedUpdate(set={

"end_ts": F.col("s.updated_at"),

"is_current": F.lit(False)

})

.execute()

)

# 2. Insert new versions for inserts/updates

inserts_df = silver_df.filter("op IN ( 'I', 'U')").select(

"customer_id", "customer_name", "phone", "address", "updated_at"

)

(dim_delta.alias("t")

.merge(

inserts_df.alias("s"),

"t.customer_id = s.customer_id AND t.start_ts = s.updated_at"

)

.whenNotMatchedInsert(values={
```

"customer_id": "s.customer_id",

"customer_name": "s.customer_name",

"phone": "s.phone",

"address": "s.address",

"start_ts": "s.updated_at",

"end_ts": F.lit(None).cast("timestamp"),

"is_current": F.lit(True)

})

.execute()

)

You can refine, but this is good enough for interviews and ebook.

6. Use cases in interview

Regulatory reporting: need full history of customer data

Audits: “What did we know about this customer on date X? ”

ML: training on historical changes

■ PRO JECT 3: Finance Analytics Lakehouse (Batch + Incremental)

1. Summary

You’re building a Lakehouse for financial analytics:

Source: multiple systems (transactions, loans, cashflows)

Landing → Bronze

Cleaning & FX normalization → Silver

Star-schema facts /dims → Gold

Dashboards: revenue, risk, cashflow, NPA, etc.

2. Example source transaction schema

txn_id | account_id | customer_id | txn_ts | amount | currency | fx_rate | region

---------- | ---------- | ---------- | ---------- | ------ | -------- | ------- | ------

TXN001 | ACC001 | CUST01 | 2025 -11-23 10:05:01 | 5000 | INR | 0.012 |

APAC

3. Bronze ingestion

bronze_path = "abfss://raw@<storage>.dfs.core.windows.net/finance/transactions /bronze"

df_bronze = (spark.read

.format("csv")

.option("header", "true")

.load(bronze_path)

)

df_bronze.write.format("delta").mode("append").saveAsTable("fin_txn_bronze")

4. Silver transformation (FX normalization, cleaning)

from pyspark.sq l import functions as F

df_bronze = spark.table("fin_txn_bronze")

df_silver = (df_bronze

.withColumn("txn_ts", F.to_timestamp("txn_ts"))

.withColumn("txn_date", F.to_date("txn_ts"))

.withColumn("amount", F.col("amount").cast("double"))

.withColumn("fx_rate", F.col("fx_rate").cast("double"))

.withColumn("amount_usd", F.col("amount") * F.col("fx_rate"))

.filter(F.col("amount").isNotNull())

)

df_silver.write.format("delta").mode("overwrite").saveAsTable("fin_txn_silver")

5. Gold model – Fact & Dimensions

Dimension tables

Example dim_date:

CREA TE TABLE IF NOT EXISTS dim_date (

date_key INT,

date_value DA TE,

year INT,

month INT,

day INT,

month_name STRING

) USING delta;

Example d im_region:

CREA TE TABLE IF NOT EXISTS dim_region (

region_key INT,

region STRING

) USING delta;

You can either hardcode dims or populate from Silver.

Fact table: fact_transactions

Schema:

txn_key (optional)

date_key

region_key

account_id

customer_id

amount_usd

txn_type (optional if available)

Gold creation (simple version):

```
df_silver = spark.table("fin_txn_silver")

df_fact_txn = (df_silver

.select(

F.col("txn_ts").alias("txn_ts"),

F.col("txn_date"),

"region ",

"account_id",

"customer_id",

"amount_usd"

)

)

df_fact_txn.write.format("delta").mode("overwrite").saveAsTable("fact_transactions")
```

6. Gold aggregations – KPIs

Example 1: Daily revenue by region

```
df_fact = spark.table(" fact_transactions")

df_daily_rev = (df_fact

.groupBy("txn_date", "region")

.agg(

F.sum("amount_usd").alias("total_revenue_usd"),

F.count("*").alias("txn_count")

)
```

)

df_daily_rev.write.format("delta").mode("overwrite").saveAsTa ble("fact_daily_revenue")

Example 2: Top N customers by revenue

SELECT customer_id,

SUM(amount_usd) AS total_revenue

FROM fact_transactions

GROUP BY customer_id

ORDER BY total_revenue DESC

LIMIT 10;

7. Incremental loading (improving the project)

Instead of overwriting:

Use a watermark / max(txn_ts) table

On each run:

Load only files after last processed timestamp

Append to Silver & Gold

Use Delta CDF to recalc only affected days